

Generative AI using Large Language Models (LLMs)

Lecture 6 | 02/27/25

Instructor: Chen Wang
Fri 4:10 pm - 6:40 pm | 517 Hamilton Hall



Important Updates & Notices

- **Proposal Relevance Requirement**

- If your final project is not relevant to your proposal topic, or it is completely different, your entire team will lose all proposal credits (10%).
- **Make sure your final deliverables align with what you proposed!**

- **Team Size Policy**

- We encourage 3 persons per team (strongly recommended).
- If you insist on 4 persons, 2 persons, or even 1 person:
 - Your collaboration score will be penalized.
 - For 4-person teams: you must meet higher bars compared to your peers.
 - Exception: If only 1-2 teams have 2 persons (due to class size modulo not being 3), it is acceptable.



TA Feedback & Updated Grading

- **TA Discussion Feedback**

- When you have team discussions with TA, the TA will give you suggestions and feedback.
- **If those feedback items are taken into account in your final report or demo, they will count toward your score.**
- **Tip: Document the TA's suggestions and show how you addressed them!**

- **Updated Grading Breakdown**

- Github Repo percentage lowered to 10%
- Demo percentage increased to 20%
- Presentation remains at 30%

- **Five Components: Proposal (10%) | Presentation (30%) | Final Report (30%) | Demo (20%) | Github Repo (10%)**



Overview

- This project will be evaluated across five main components:
 - Project Proposal (10%)
 - Final Project Presentation (30%)
 - Final Report (30%)
 - Project Demo (20%)
 - Github Repo (10%)



Project Timeline

- **March 10, 2026**
 - Project Proposal deadline
- **March 10 – March 30**
 - Team discussions with TA, Christopher Ng (cn2673@columbia.edu)
- **May 3, 2026**
 - Final Project Presentation and Report



Project Proposal (10%)

Clarity of Objectives (4%)

- Clear definition of research questions or development goals
- Specific problems to be addressed
- Expected outcomes and deliverables
- Alignment with course objectives

Feasibility (3%)

- Realistic timeline and milestones
- Assessment of required resources and their availability
- Identification of potential challenges and mitigation strategies
- Clear scope definition

Innovation and Relevance (3%)

- Originality of the proposed approach
- Connection to current GenAI research trends
- Potential impact on the field
- Novel application or methodology



Final Project Presentation (30%)

Problem Statement (10%)

Context and Background

Clear description of the problem domain

Current state of the art and its limitations

Significance of the problem in GenAI context

Target audience and use cases

Research Questions

Clearly articulated research questions

Hypotheses and assumptions

Scope and boundaries of the investigation



Final Project Presentation (cont.)

**Major
Contributions
(10%)**

Technical Innovations

Novel algorithms or methodologies

Architectural improvements

Performance enhancements

Integration strategies

Practical Impact

Real-world applications

Scalability considerations

Resource efficiency

User experience improvements



Final Project Presentation (cont.)

**Evaluation
(10%)**

Quantitative Metrics

Performance measurements

Comparative analysis with baselines

Statistical significance of results

Ablation studies

Qualitative Analysis

Case studies

User studies (if applicable)

Error analysis

Limitations and future work



Final Report (30%)

Structure and Format (5%)

Professional layout and formatting

Logical flow of sections

Proper citations and references

Clear figures and tables

Depth of Research (8%)

Comprehensive literature review

Critical analysis of related work

Understanding of fundamental concepts

Integration of multiple perspectives



Final Report (cont.)

Methodology (8%)

- Detailed experimental setup
- Data collection and preprocessing
- Model architecture and implementation
- Training and optimization procedures

Results and Analysis (6%)

- Clear presentation of findings
- Appropriate visualizations
- Thorough analysis of results
- Discussion of implications

Grammar and Writing Quality (3%)

- Professional academic tone
- Clear and concise writing
- Proper technical terminology
- Consistent formatting



Project Demo (20%)

Code Quality (10%)

Clean and organized code structure

Comprehensive documentation

Unit tests and error handling

Code optimization

Functionality (10%)

Successful implementation of proposed features

Robust error handling

Performance optimization

Integration testing



Github Repo (10%)

Experiment Results (8%)

Reproducible experiments

Well-documented experimental setup

Clear presentation of results

Analysis scripts and notebooks

Step-by- Step Tutorial (2%)

Clear installation instructions

Environment setup guide

Usage examples and demonstrations

Troubleshooting guide

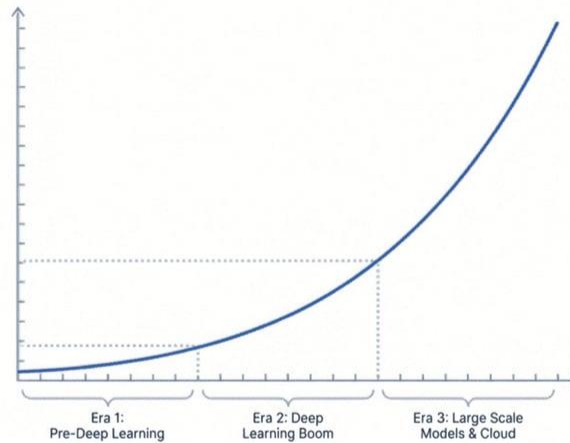


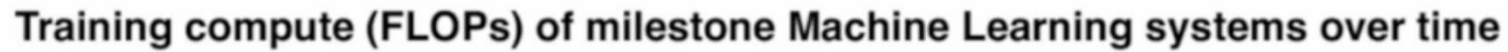
Grading Summary

Component	Percentage
Project Proposal	10%
Final Project Presentation	30%
Final Report	30%
Demo	20%
Github Repo	10%

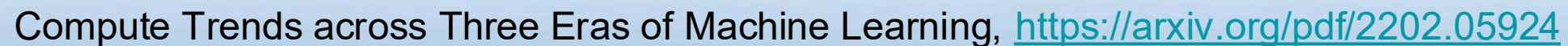
Compute Trends Across Three Eras of Machine Learning

Understanding AI's Exponential Compute Demand and the Case for Cloud-Scale Infrastructure

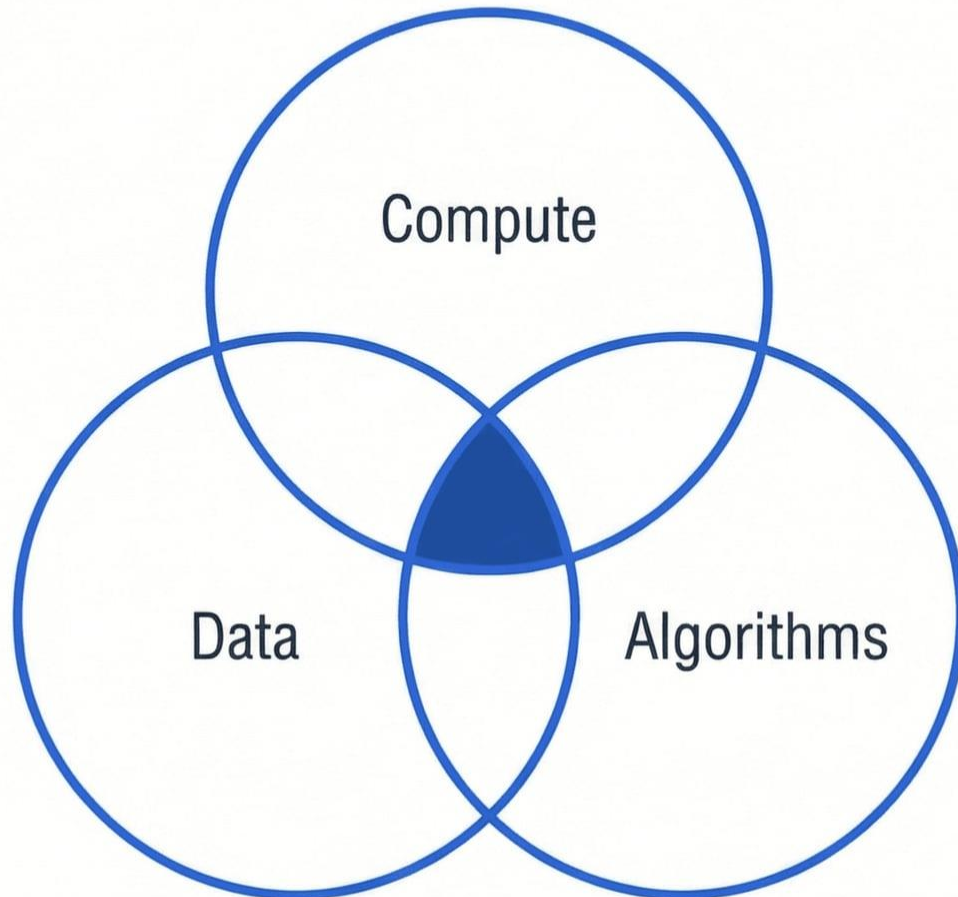




2026

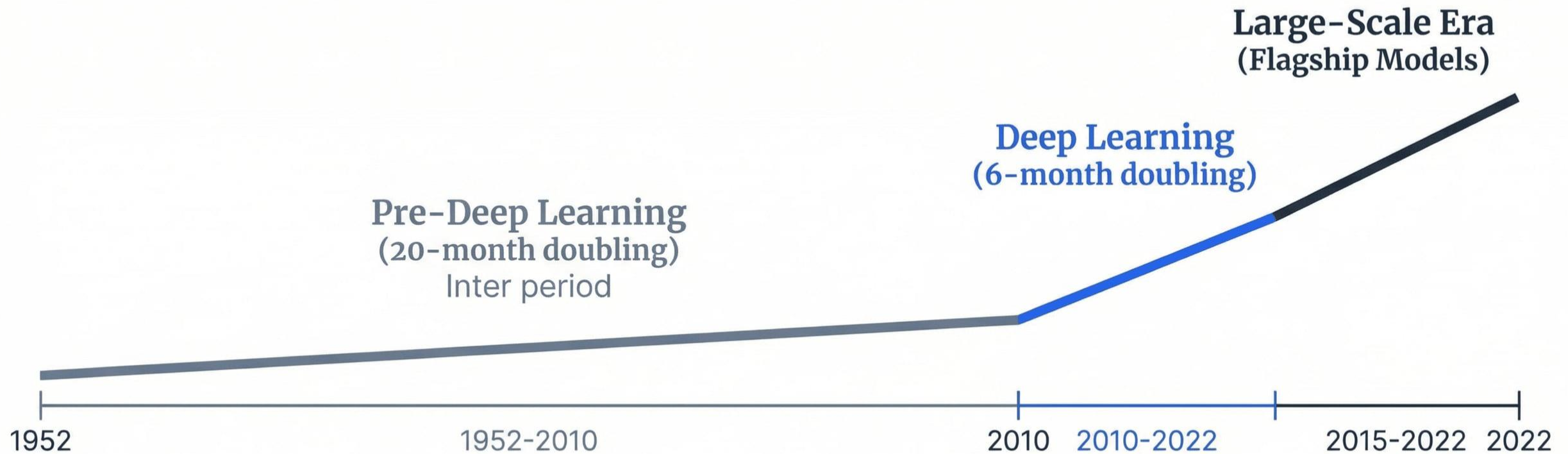


AI Progress Is Driven by Three Pillars: Compute, Data, and Algorithms



- Compute is the most quantifiable driver of ML progress.
- Training compute has grown exponentially, far outpacing Moore's Law since 2010.
- Understanding compute trends is critical for infrastructure planning.
- Cloud AI market exceeded \$100B in 2025, projected to surpass \$1T by 2034.

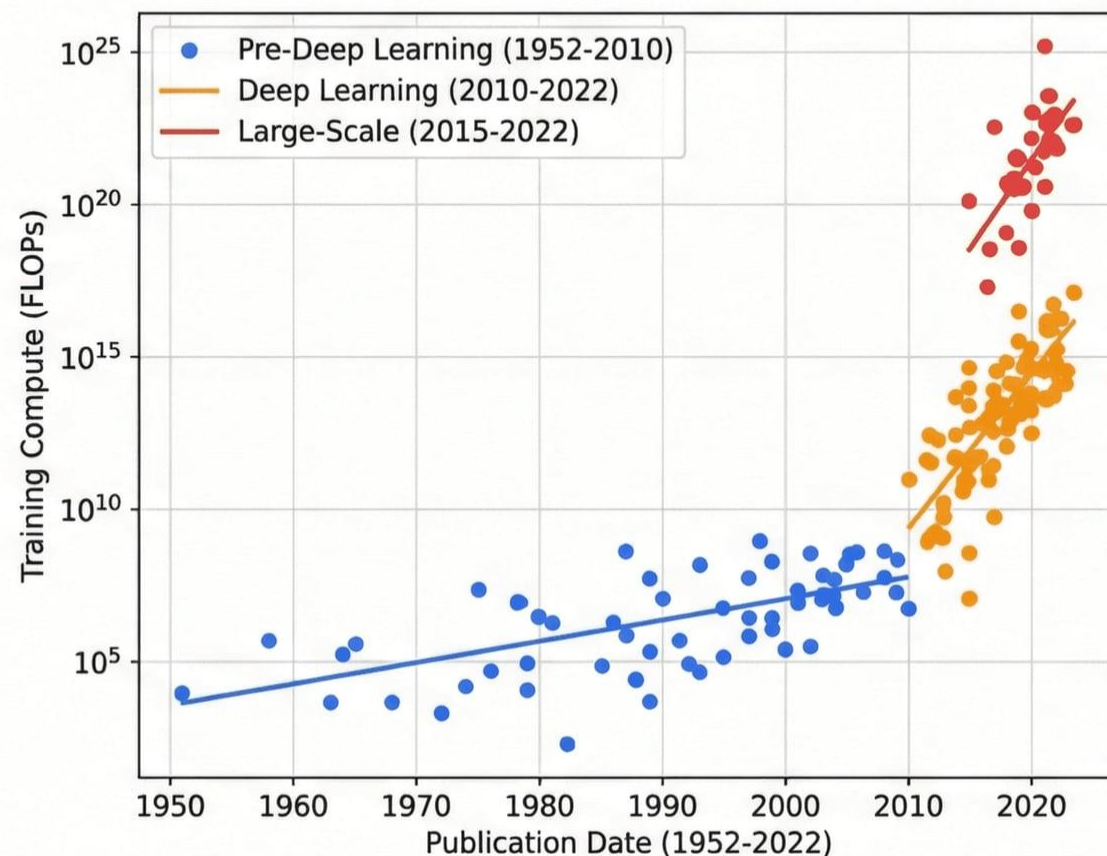
Seven Decades of ML: Compute Grew 10 Billion-Fold



- Pre-DL Era: Matches Moore's Law (~20 months)
- DL Era: Accelerates to ~6 months (4x faster)
- Total Growth: From 10^4 to 10^{24} FLOPs

‘Training Compute (FLOPs) of Milestone ML Systems Over Time (1952–2022)’

- Figure 1: 123 milestone ML systems plotted on log-scale FLOPs vs. time
- Three distinct trend lines visible: Pre-DL, DL regular-scale, and Large-Scale
- Statistical methodology: log-linear regression with bootstrap CIs ($B=1000$)
- Z-score filtering: outliers ($Z < -2$), regular-scale ($-2 < Z < 0.76$), large-scale ($Z \geq 0.76$)
- This is the paper's Figure 1 — the key visualization to understand



Doubling Times Tell the Story: 20 Months → 6 Months → 10 Months

Pre-DL Era
(1952-2010)

21.3 Months

Matches Moore's Law

Deep Learning Era
(2010-2022)

5.7 Months

4× Faster than Moore's Law

Large-Scale Era
(2015-2022)

9.9 Months

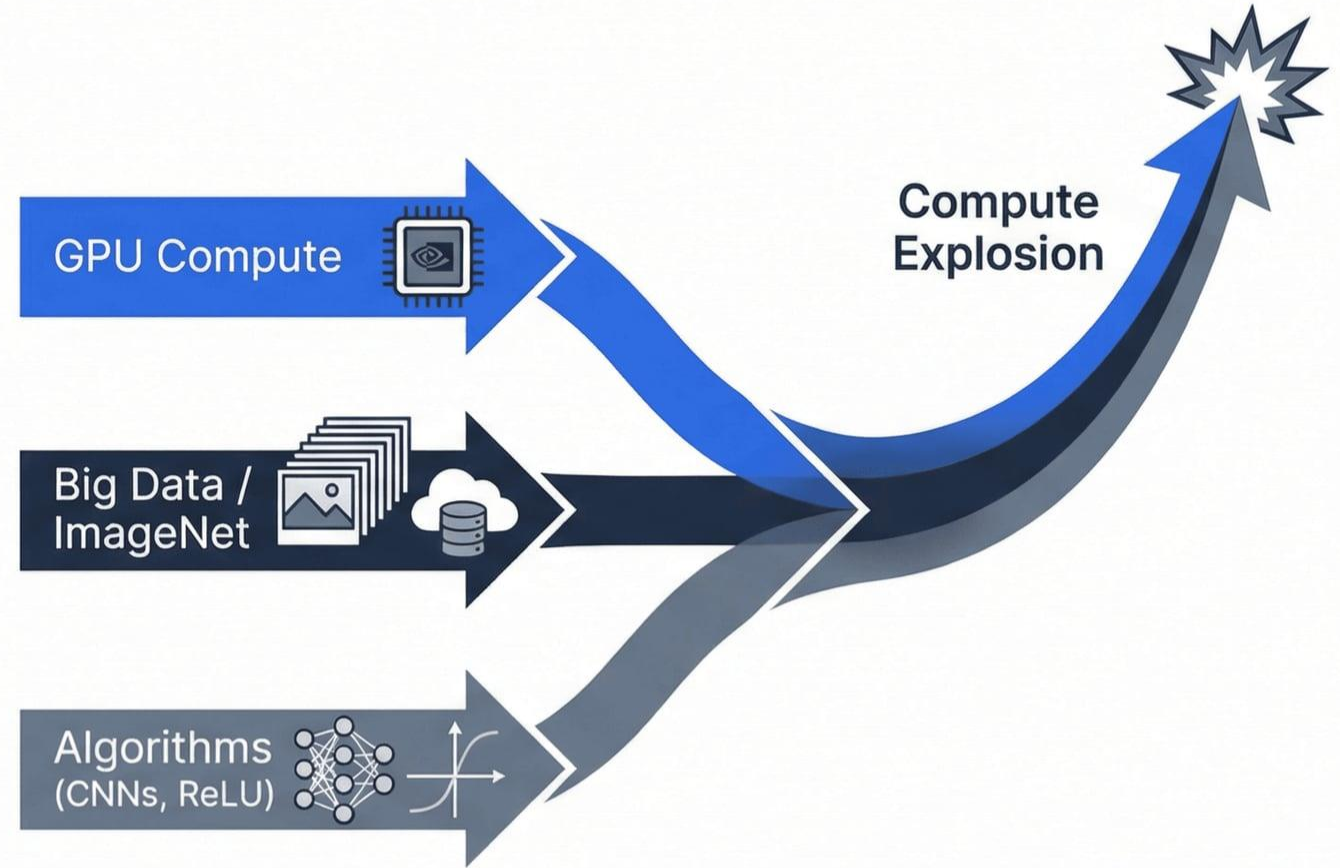
From 100× Higher Baseline

The gap between compute demand (5-6 mo doubling) and hardware improvement (24 mo doubling) creates the scaling challenge.

The Deep Learning Inflection

GPUs, ImageNet, and Architectural Innovation Triggered the Compute Explosion

- GPU training matured ~2009–2011 (10–100× compute boost)
- ImageNet (2009) provided the data fuel for deep models
- Architectural innovations (CNNs, ReLU) enabled effective scaling
- The transition began ~2010, predating AlexNet (2012)



The Large-Scale Era

From AlphaGo to GPT-3: When Corporations Began Spending \$10M+ on Single Training Runs

- Era began ~late 2015 with AlphaGo (DeepMind)
- Flagship models follow a distinct, steeper growth trajectory (10-month doubling)
- Represents a qualitative shift in investment strategy
- As of 2025: US controls ~75% of global AI compute

Estimated Training Costs (Flagship Models)



Domain-Specific Trends: Language Models Lead the Pack

Language Models

0.7 OOMs/year

**4.8-month
doubling**

Computer Vision

0.6 OOMs/year

**6.0-month
doubling**

Games / RL

0.8 OOMs/year

**4.5-month
doubling**
(High Variance)

Speech / Robotics

0.5 OOMs/year

**6.8-month
doubling**

Compute scaling is universal across ML domains, driven by economic value and investment.

The Methodology — How to Measure Training Compute

Estimating FLOPs: From GPU-Hours to Floating Point Operations

Dataset & Analysis

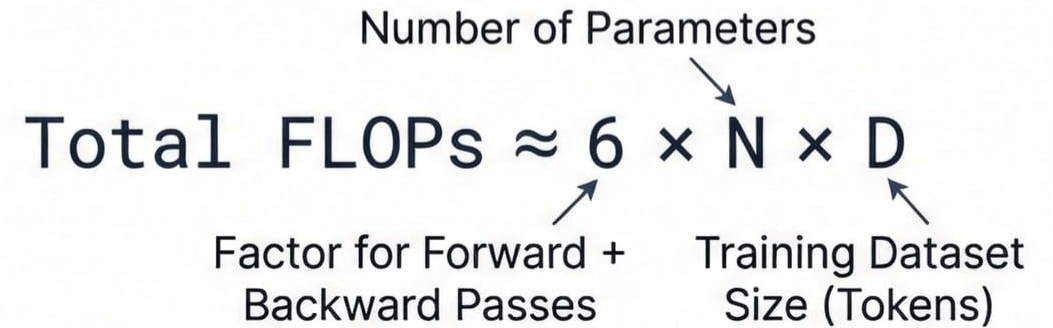
- **Metric:** Total training FLOPs (floating-point operations)
- **Dataset:** 123 milestone ML systems (1952–2022)
- **Selection:** Explicit learning, SotA results, historical significance
- **Statistical Method:** Log-linear regression with bootstrap CIs (B=1000)
- **Filtering:** Z-score separation of regular vs. large-scale models

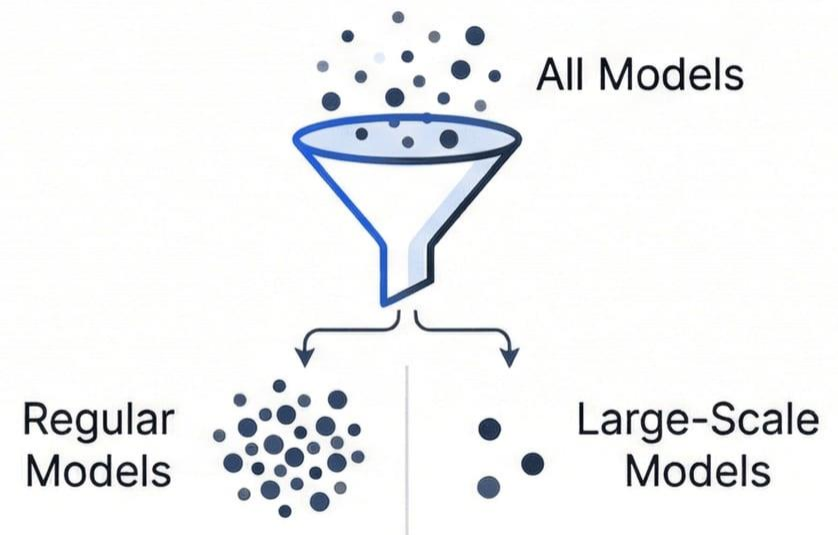
$$\text{Total FLOPs} \approx 6 \times N \times D$$

Number of Parameters

Factor for Forward + Backward Passes

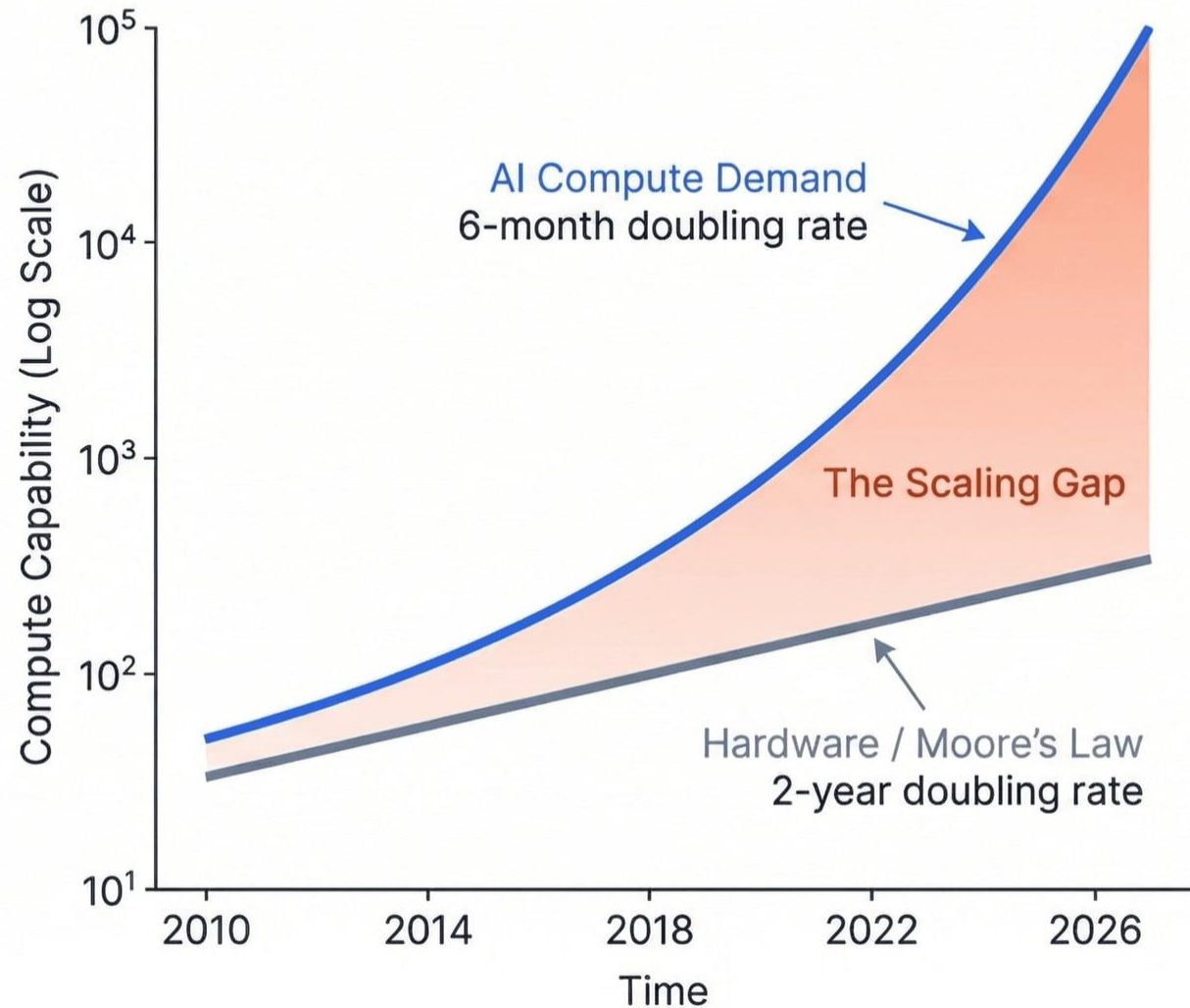
Training Dataset Size (Tokens)





The Compute Gap: AI Demand vs. Hardware Reality

AI Compute Demand Grows 4× Faster Than Hardware Improves



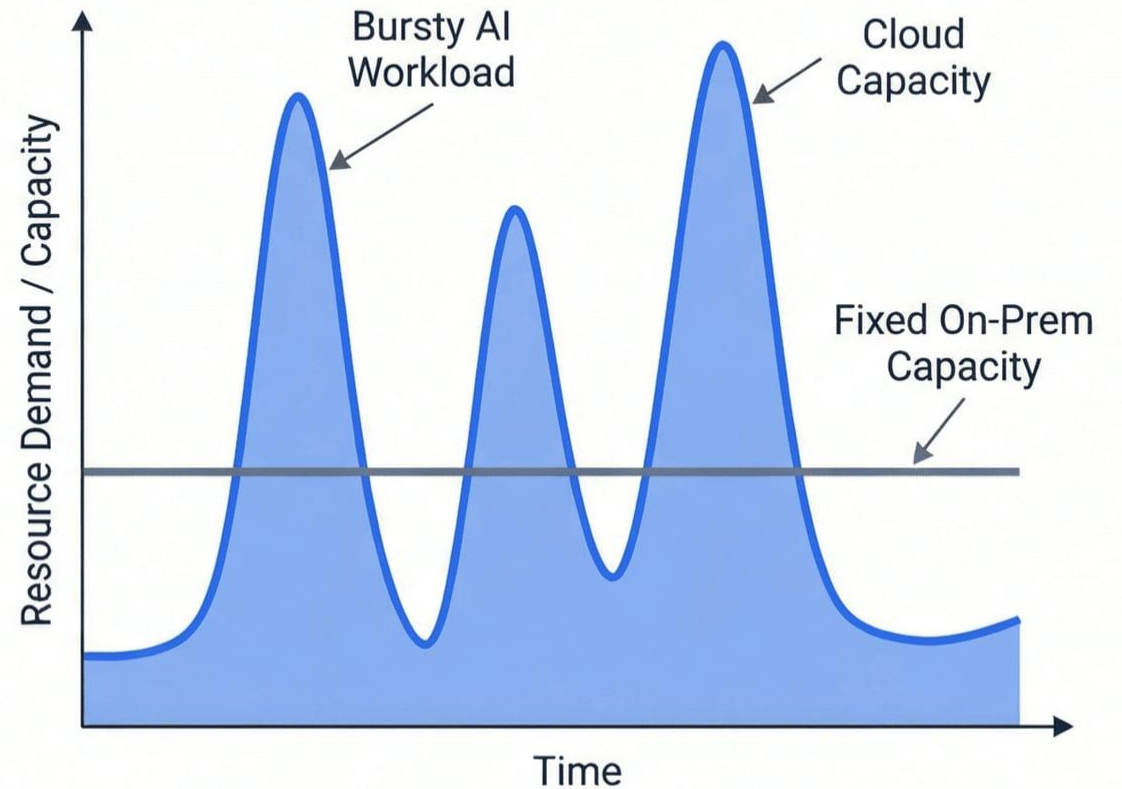
Key Metrics

- **Moore's Law:** ~2-year doubling of hardware capability.
- **AI Demand:** ~5-6 month doubling (4× faster).
- **Efficiency:** Hardware improves 37%/year vs. 4-5× demand growth.
- **Consequence:** Exponentially increasing costs for frontier research.
- **Solution:** No single organization can own enough hardware — cloud is the answer.

Why Cloud? — Elastic Infrastructure for Exponential Demand

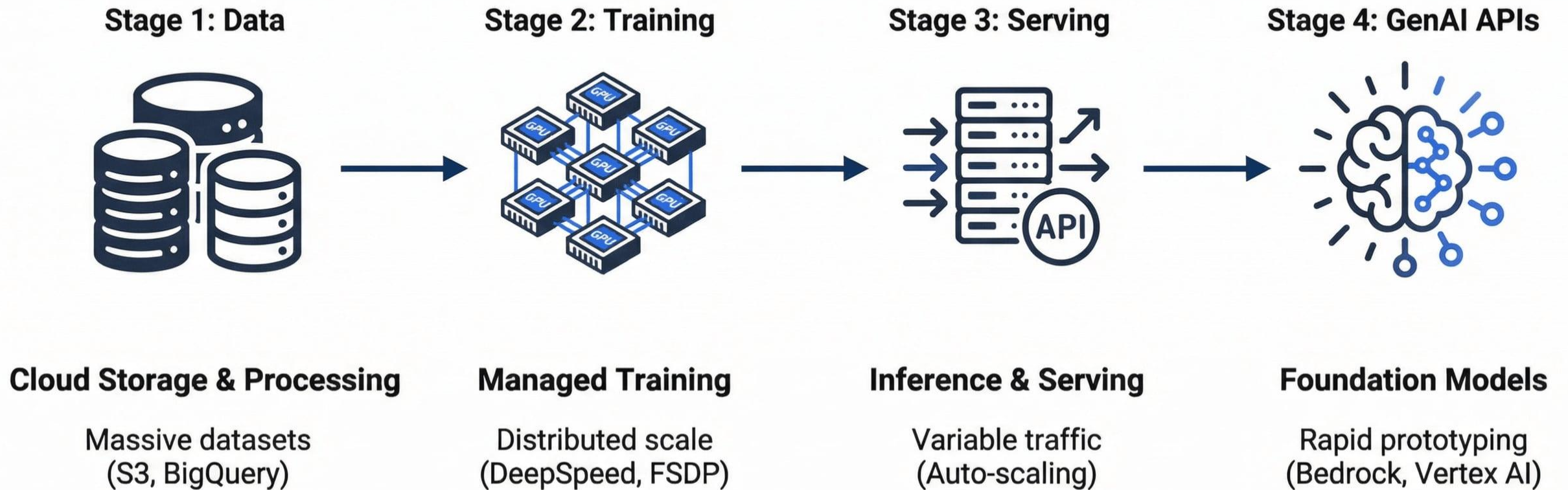
Key Benefits

- **Elasticity:** Scale from 0 to 1000s of GPUs on demand; pay only for usage.
- **Hardware Access:** Immediate access to latest accelerators (H100, TPUs) without CapEx.
- **Managed Services:** Abstract complexity via SageMaker, Vertex AI, Azure ML.
- **Economics:** Shift from CapEx to OpEx; avoid low utilization of owned clusters.



From Paper to Practice — Building GenAI Applications on Cloud

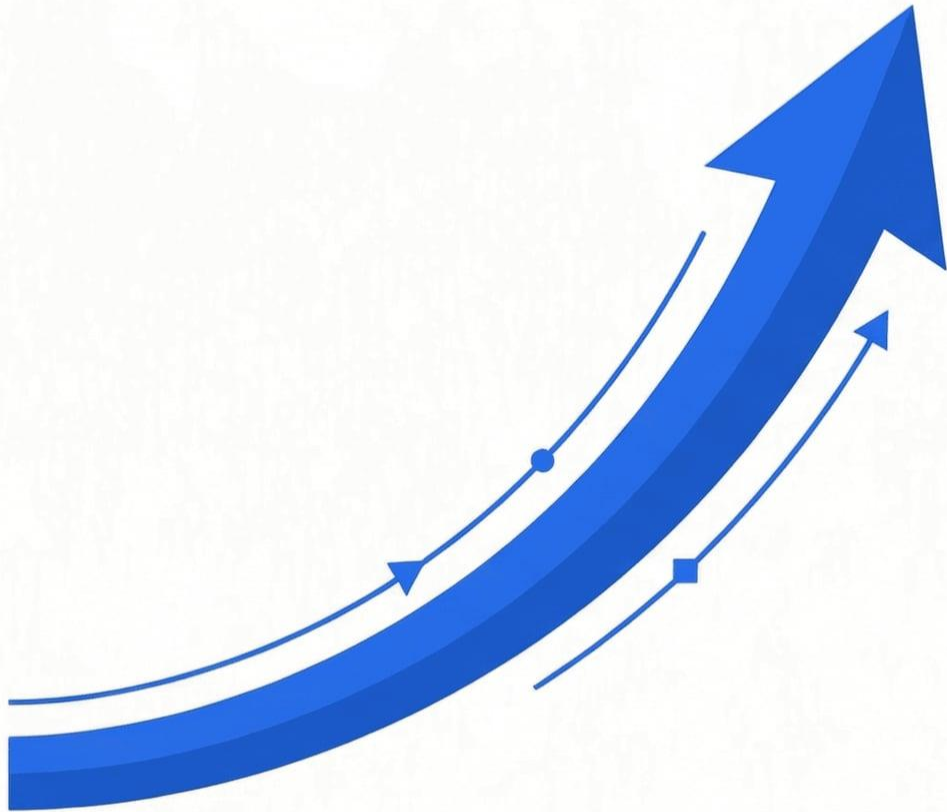
Cloud Enables the Full ML Lifecycle: From Data to Deployment at Scale



The compute trend means cloud infrastructure is a necessity, not a luxury, for AI development.

Key Takeaways: The Exponential Compute Trend Continues

Prepare for a Cloud-Native AI Future



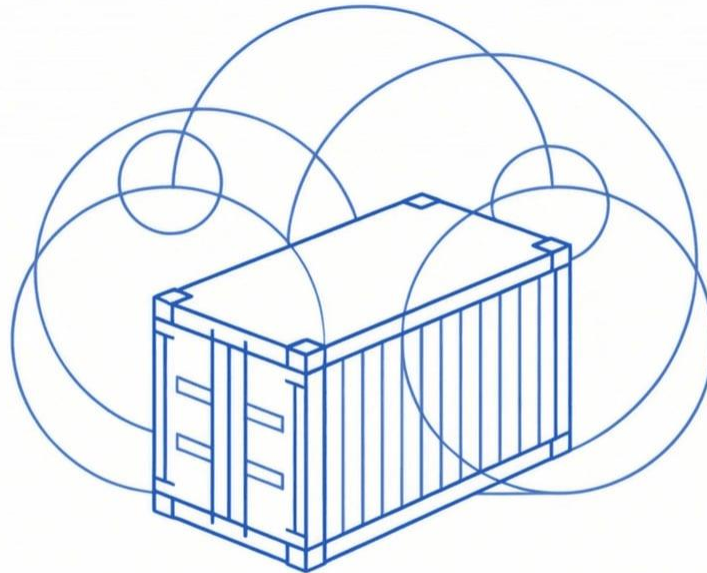
- **Three Eras:** Pre-DL (20-mo), DL (6-mo), Large-Scale (10-mo doubling).
- **Unrelenting Growth:** Compute demand continues at 4–5× per year.
- **Escalating Costs:** Frontier training runs now exceed \$100M.
- **Cloud Necessity:** Ownership economics favor cloud for all but hyperscalers.
- **Core Competency:** Cloud ML infrastructure mastery is essential for GenAI.

The future of AI is cloud-native.

Containers, Docker, Kubernetes & Google Cloud Run

COMSE6998-015 Spring 2026 — Introduction to LLM-based Generative AI Systems

Deploying a Gemini-Powered Chat App in the Cloud



Resource Management: The Foundation of Scalable AI Deployment

Generative AI teams increasingly share powerful GPU clusters, leading to challenges with conflicting software dependencies and resource contention among multiple users and projects.

Function: Multi-tenancy



Shared GPUs & diverse environments

Goal: Isolation



Reproducible experiments & allocation

Solutions (2026)



Docker, K8s, Cloud Run, Ray

Docker + Kubernetes: The Container Ecosystem at a Glance

Docker Builds the Containers, Kubernetes Orchestrates Them at Scale

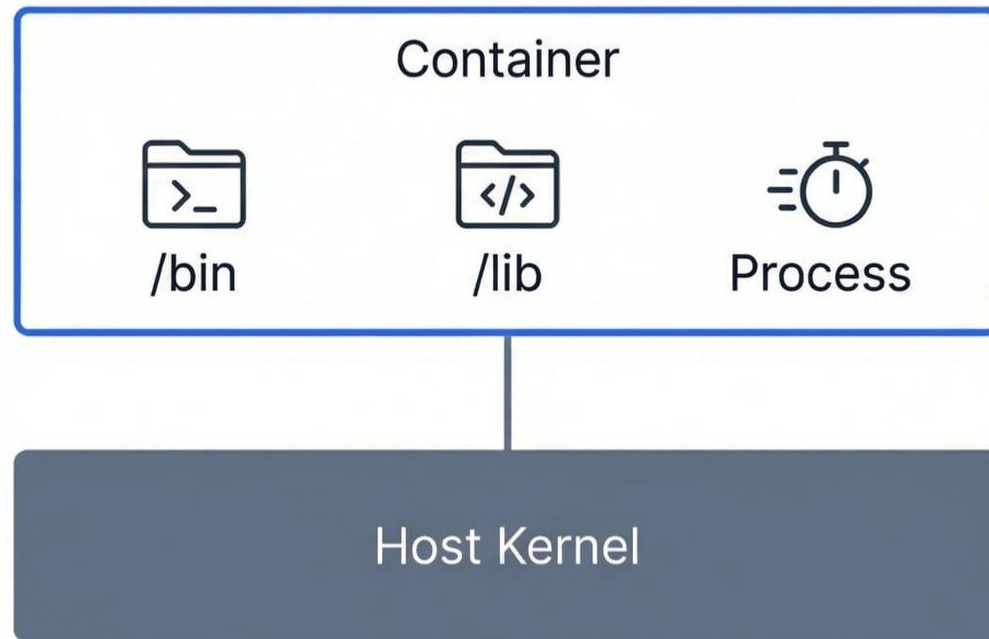


Docker packages code + dependencies into portable images.

Kubernetes orchestrates containers across clusters (scheduling, scaling, self-healing).

2026 Standards: Docker Engine v29 & K8s 1.32+

Containers Leverage Linux Kernel Primitives for Lightweight Isolation



Linux Namespaces (Isolation)

- PID
- Network
- Mount
- UTS



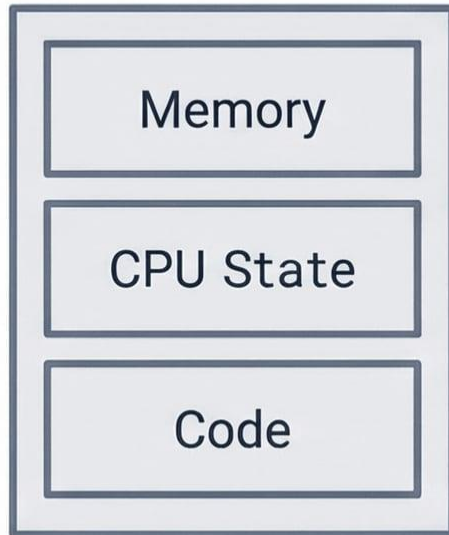
Linux Cgroups (Resource Limits)

- CPU
- Memory
- Disk I/O

- Result: Lightweight, Fast Startup, High Density

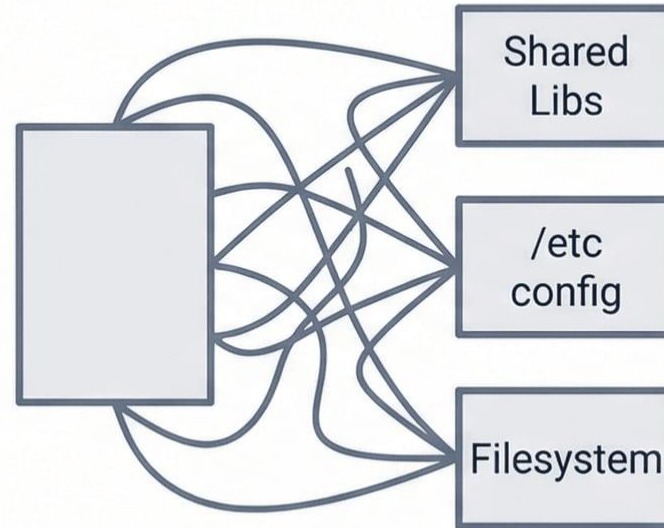
Containers vs. Processes: From Textbook to Encapsulated

Textbook Process



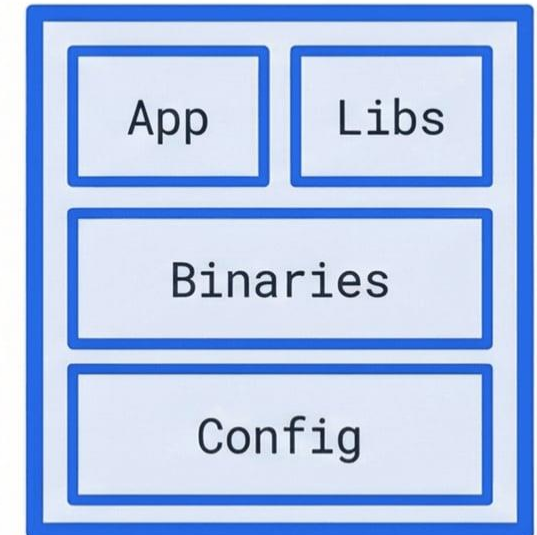
Isolated address space,
simple abstraction.

Real Process



Dependent on host OS,
shared libraries, conflicts.

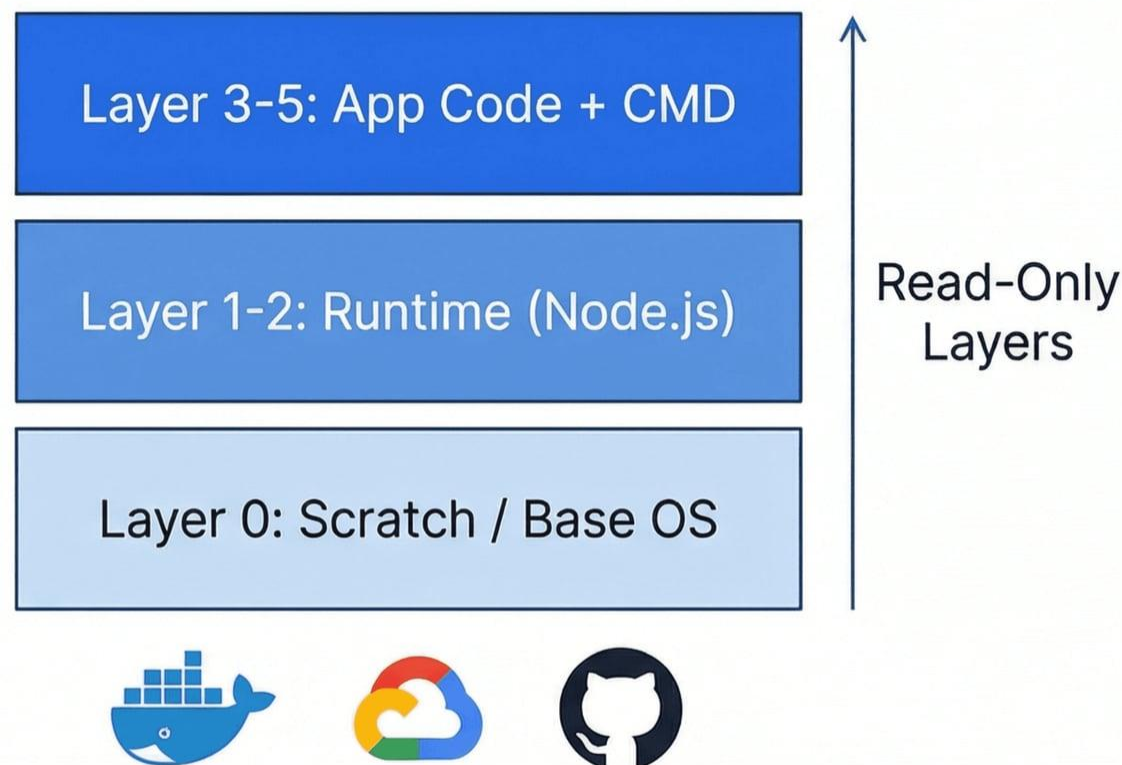
Container



Encapsulated dependencies,
portable, reproducible.

Docker Simplifies Container Lifecycle: Build, Ship, and Run Anywhere

- **Docker image:** Multi-layered read-only template (each layer = one instruction)
- **Dockerfile:** FROM base → COPY code → RUN install → CMD start
- **Registries (2026):** Docker Hub, Google Artifact Registry, GitHub Container Registry
- **2026 Updates:** Docker Engine v29, Free Hardened Images, Gordon AI Agent (Beta)



Multi-Stage Builds and Minimal Base Images Are the 2026 Standard

Key Practices

- **Multi-stage builds:** Separate build tools from runtime.
- **Minimal base images:** Use `distroless` or `slim` variants.
- **Layer ordering:** Optimize for caching (dependencies first).
- **Security:** Run as non-root user, pin versions.
- **Result:** 88% size reduction (1.2GB → 150MB).

Stage 1: Builder

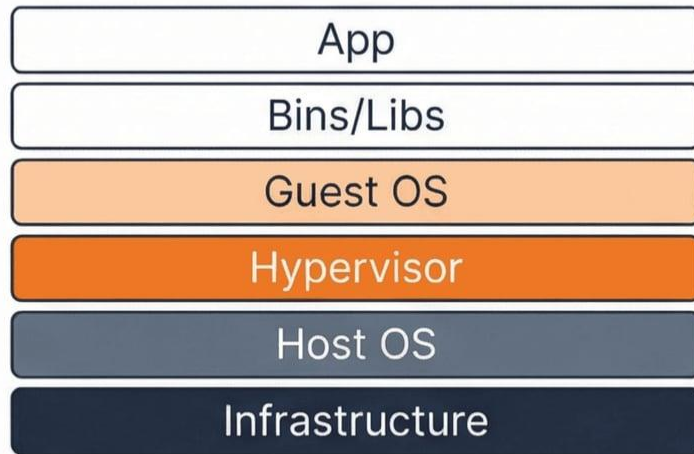
```
FROM golang:1.26-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o myapp .
```

Stage 2: Production

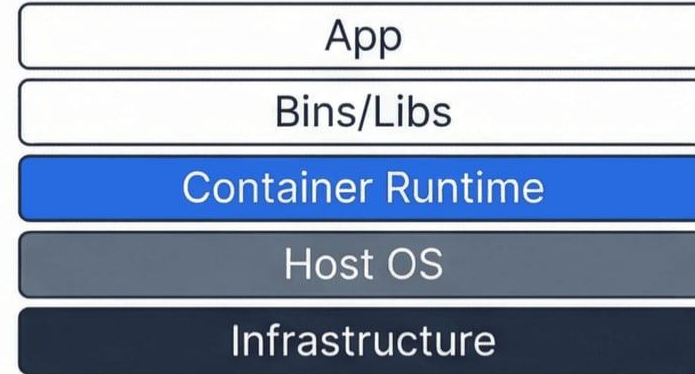
```
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=builder /app/myapp /
USER nonroot:nonroot
ENTRYPOINT ["/myapp"]
```

Containers vs. Virtual Machines: Architectural Differences

Virtual Machine (Heavyweight)



Container (Lightweight)



Feature	Virtual Machines (VMs)	Containers
Size	GBs	MBs
Boot Time	Minutes	Seconds
Isolation	Hardware	Kernel

Benefits of Containerization for GenAI Development Teams

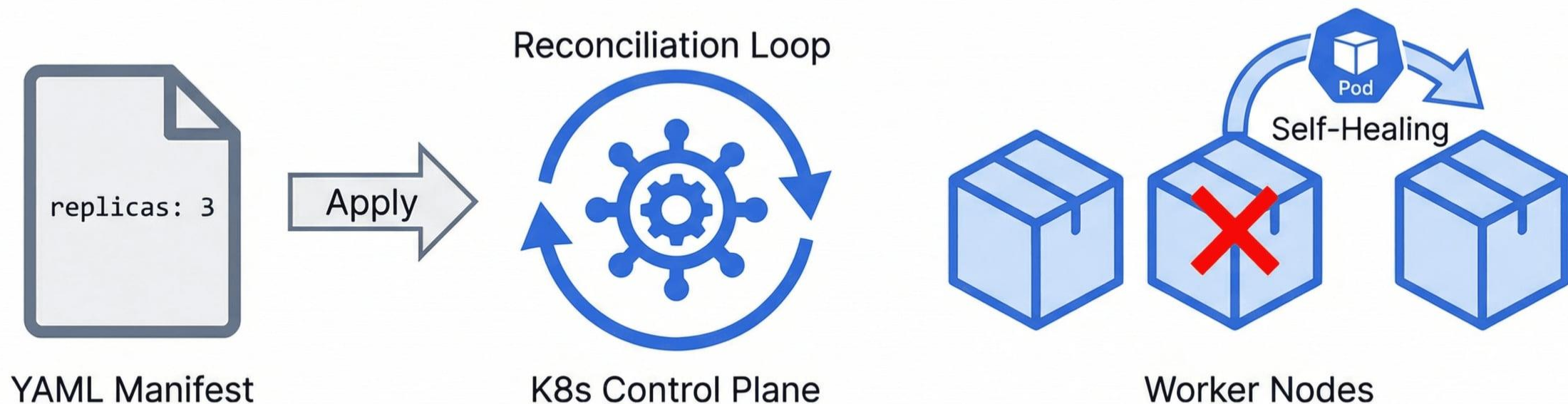
Containers Enable Speed, Consistency, and Scale for AI Workloads

Key Benefits for Development Teams

- ➡ **Move Fast:** Environment decoupling, rapid iteration.
- 📄 **Deploy Efficiently:** Consistent dev/prod environments, run anywhere.
- 📉 **Operate at Scale:** Auto-scaling, self-healing with K8s.
- ⚖️ **GenAI Specifics:** Reproducible model environments, NVIDIA GPU support, model versioning.

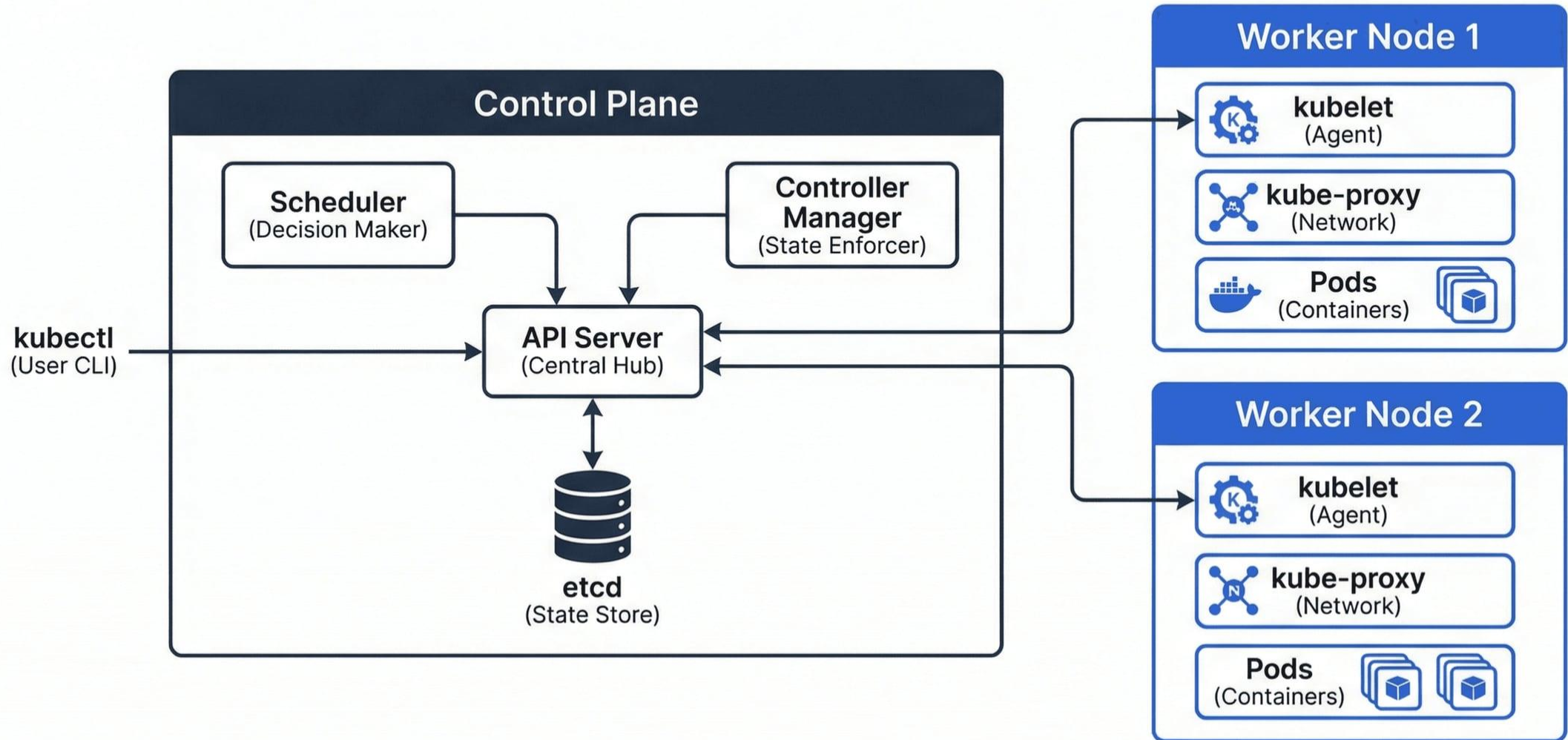
Containers vs. Virtual Machines Comparison		
	Containers	Virtual Machines
Consistent Runtime	✓	✓
Sandboxing	✓	✓
Small Size (MBs)	✓	✗
Low Overhead	✓	✗

Kubernetes Declares What You Want and Continuously Reconciles Reality



- **Desired State:** Declare in YAML → K8s reconciles reality.
- **Self-Healing:** Automatic rescheduling on node failure.
- **API Server:** Central hub for all cluster communication.

Kubernetes Architecture: Master-Worker Pattern with Specialized Components



Managed Kubernetes Services Eliminate Cluster Operations Overhead (2026 Landscape)



Google (GKE)

Pioneer of K8s, runs YouTube/Gmail. GKE Autopilot mode.



AWS (EKS)

Largest market share. Integrated with Fargate.



Azure (AKS)

Tight integration with Azure DevOps & Active Directory.

Serverless Alternatives
No Cluster Management

Google Cloud Run, AWS Fargate, Azure Container Apps

Cloud Run Gives You Container Flexibility with Serverless Simplicity



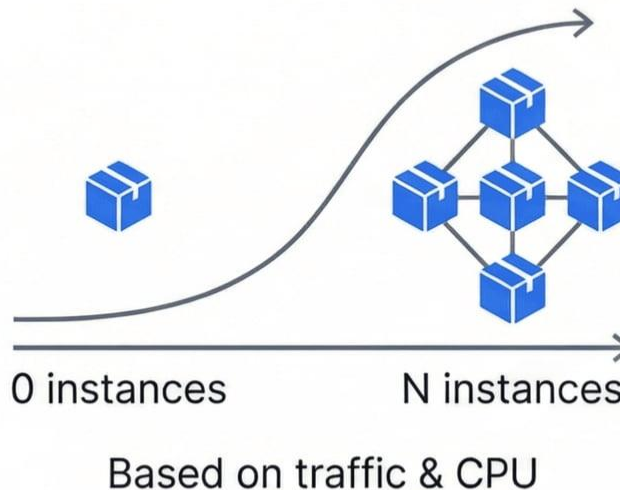
Source Code



Container Image



Cloud Run
Managed Service



HTTPS
Endpoint



Deploy:
``gcloud run deploy``
(Simple CLI)



Auto-scaling:
Scale to Zero
(Cost efficiency)



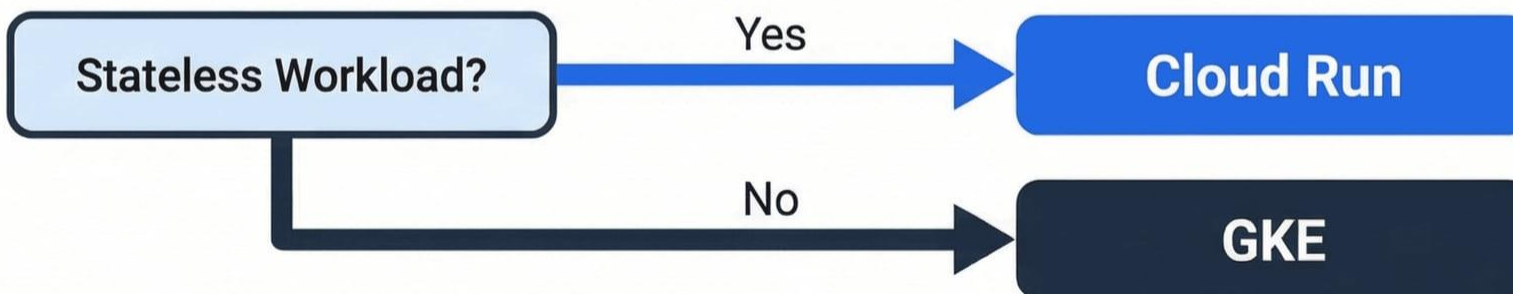
GPU Support (2026):
NVIDIA L4 for AI
Inference



Pay-per-use:
Billed per 100ms

Use Cloud Run for Simplicity, GKE for Full Kubernetes Control

Cloud Run (Serverless)	GKE (Managed K8s)
 Stateless No Ops Scale to Zero Per-request Billing	 Stateful Full Control Custom Networking GPU Pools
Best For: APIs, Web Apps, Event-driven	Best For: Databases, Legacy Apps, Complex Mesh



Introduction to Generative AI in the Cloud

Transformative Impact

Generative AI is revolutionizing industries by enabling content creation, data synthesis, and decision support at unprecedented scale and quality.

Cloud Provider Role

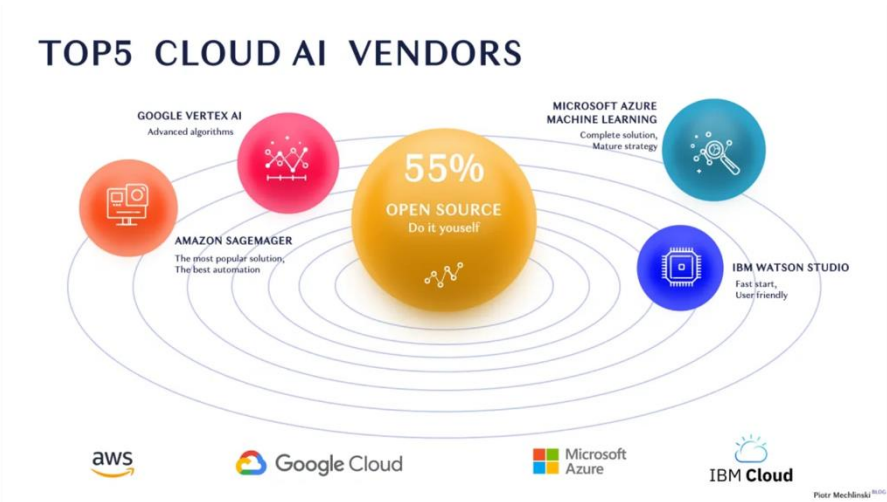
Major cloud providers are democratizing access to GenAI through managed services, reducing the technical barriers and infrastructure costs.

Enterprise Adoption

By 2025, over 80% of enterprises are integrating GenAI into their workflows, with cloud platforms serving as the primary deployment vehicle.

GenAI Category	GenAI Component	Amazon Web Services	Google Cloud	Microsoft Azure
Foundation Models	Runtime	Amazon Bedrock	Vertex AI	Azure OpenAI
	Text / Chat	TBD	PaLM	GPT
	Code	TBD	Codey	GPT
	Image Generation	TBD	Imagen	DALL-E
	Translation	TBD	Chirp	None
Model Catalog	Commercial	Amazon SageMaker JumpStart Amazon Titan	Vertex AI Model Garden	Azure ML Foundation Models
	Open Source	Amazon SageMaker JumpStart Hugging Face	Vertex AI Model Garden	Azure ML Hugging Face
Vector Database		Amazon RDS (pgvector)	Cloud SQL (pgvector)	Azure Cosmos DB Azure Cache
Model Deployment & Inference		Amazon SageMaker	Vertex AI	Azure ML
Fine-tuning		Amazon Bedrock	Vertex AI	Azure OpenAI
Low-code/No-code Development		TBD	Gen App Builder	Power Apps
Code Completion		Amazon Code Whisperer	Duet AI for Google Cloud	GitHub Copilot

Landscape of Major Cloud GenAI Platforms



Platform	Key Strengths	Flagship GenAI Services
AWS	Extensive service portfolio, global reach	SageMaker, Bedrock, Titan
Microsoft Azure	OpenAI partnership, ecosystem integration	Azure OpenAI Service, ML Studio
Google Cloud	AI research leadership, Gemini models	Vertex AI, Model Garden
IBM	Enterprise governance, hybrid cloud	Watsonx.ai, Watsonx.governance

AWS Generative AI Offerings

Amazon Bedrock

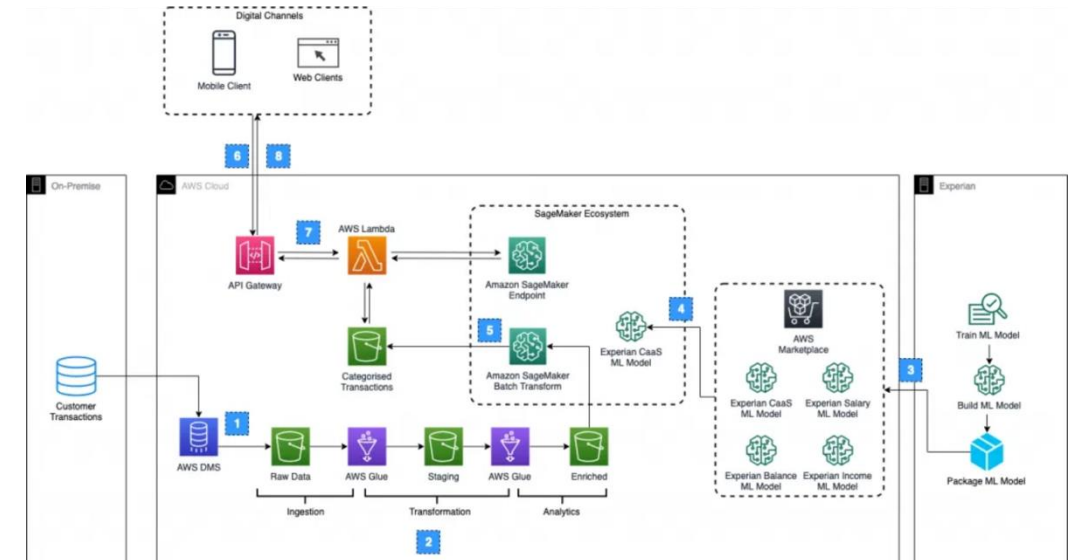
Serverless, fully-managed service providing access to foundation models through a single API.

Amazon SageMaker

End-to-end ML platform for building, training, and deploying machine learning models at scale.

Amazon Titan

AWS-built foundation models for text and multimodal applications with customization options.



Microsoft Azure AI Platform

Azure Machine Learning

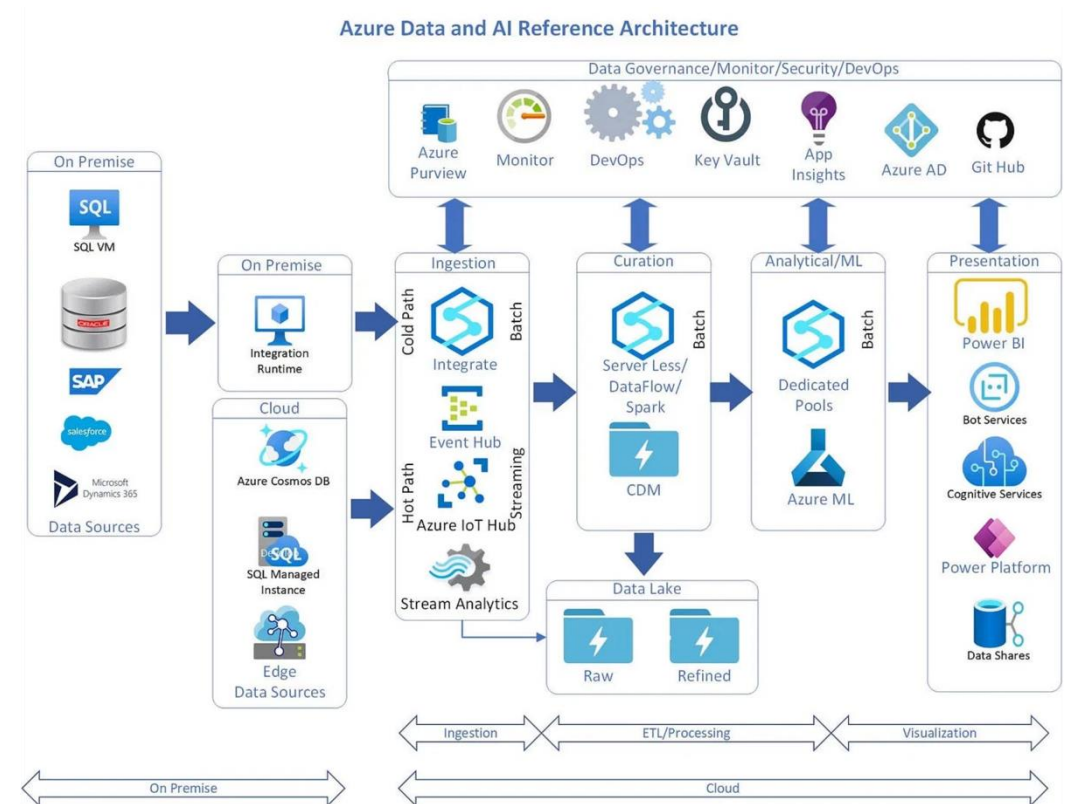
End-to-end platform for building, training, and deploying machine learning models with robust MLOps capabilities.

Azure OpenAI Service

Enterprise-ready access to OpenAI's models with added security, compliance, and regional availability.

Microsoft Ecosystem Integration

Seamless integration with Microsoft 365, Dynamics, Power Platform, and other enterprise applications.



IBM Watsonx

watsonx.ai

An AI studio for building, training, and deploying machine learning models with a comprehensive developer toolkit and lifecycle management.

watsonx.data

Data management platform that unifies and integrates data from any source to improve AI application relevance and precision.

watsonx.governance

Automates governance processes to manage AI risks, simplify regulatory compliance, and create responsible AI workflows.

- ✓ Enterprise-grade security and compliance controls
- ✓ Hybrid cloud architecture with flexible deployment options
- ✓ Strong focus on AI governance and explainability

Platform components



watsonx.ai

Build with our new studio for foundation models, generative AI and machine learning. With watsonx.ai, you can train, validate, tune and deploy foundation and machine learning models with ease.



watsonx.data

Scale analytics and AI workloads for all your data, anywhere with watsonx.data, the industry's only data store that is open, hybrid and governed.



watsonx.governance

Accelerate responsibility, transparency and explainability in your data and AI workflows with watsonx.governance. This solution helps you direct, manage and monitor your organization's AI activities. Now in Tech Preview. General availability of watsonx.governance is expected in December.

Deep Dive: Google Cloud Vertex AI - Introduction

Vertex AI is a comprehensive, unified machine learning platform that combines data engineering, data science, and ML engineering workflows, enabling teams to collaborate using a common toolset.

Unified Platform

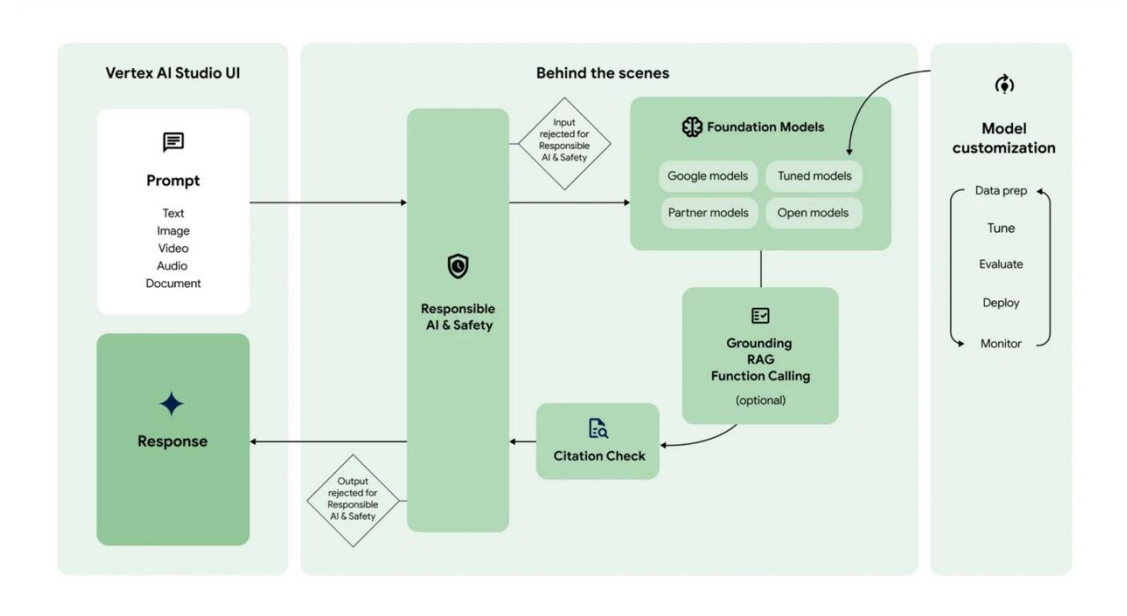
Seamlessly integrates all ML workflows from data preparation to model deployment

Foundation Models

Access to 200+ models including Google's Gemini, third-party, and open-source options

Enterprise-Ready

Robust governance, security controls, and MLOps capabilities for production deployments



Vertex AI - Model Garden and Gemini Integration



Model Garden

First-Party Models

Gemini (Pro, Ultra, Flash)
Imagen for image generation
Chirp for speech processing

Third-Party Models

Anthropic Claude Model Family

Open-Source Models

Gemma, Llama 3.2



Gemini Multimodal Capabilities

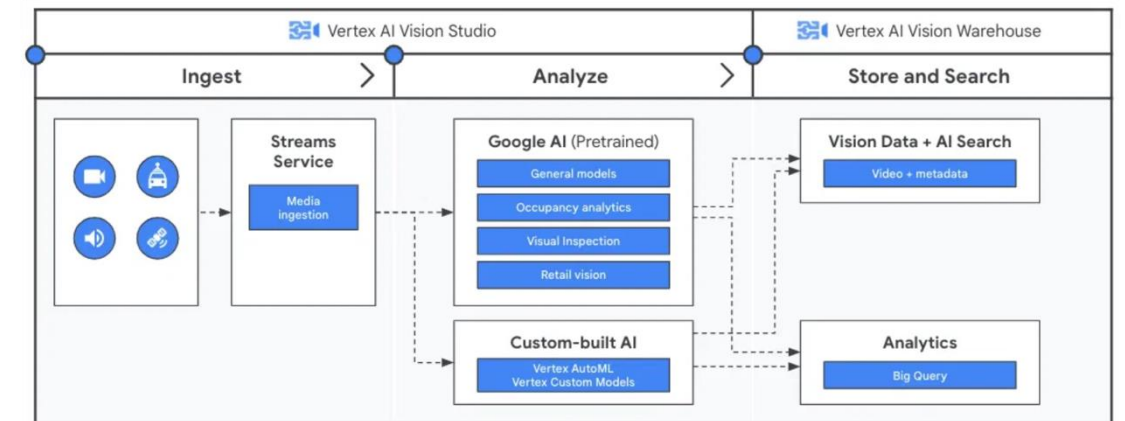
Process and understand text, images, video, and code inputs simultaneously



Advanced reasoning for complex tasks like extracting text from images and converting to structured formats



Customizable through tuning options for specific use cases and domains



Vertex AI - Training, Deployment, and MLOps

Flexible Training Options

Choose between AutoML for code-free model development or Custom Training for complete control over frameworks, code, and hyperparameters.

Vertex AI Pipelines

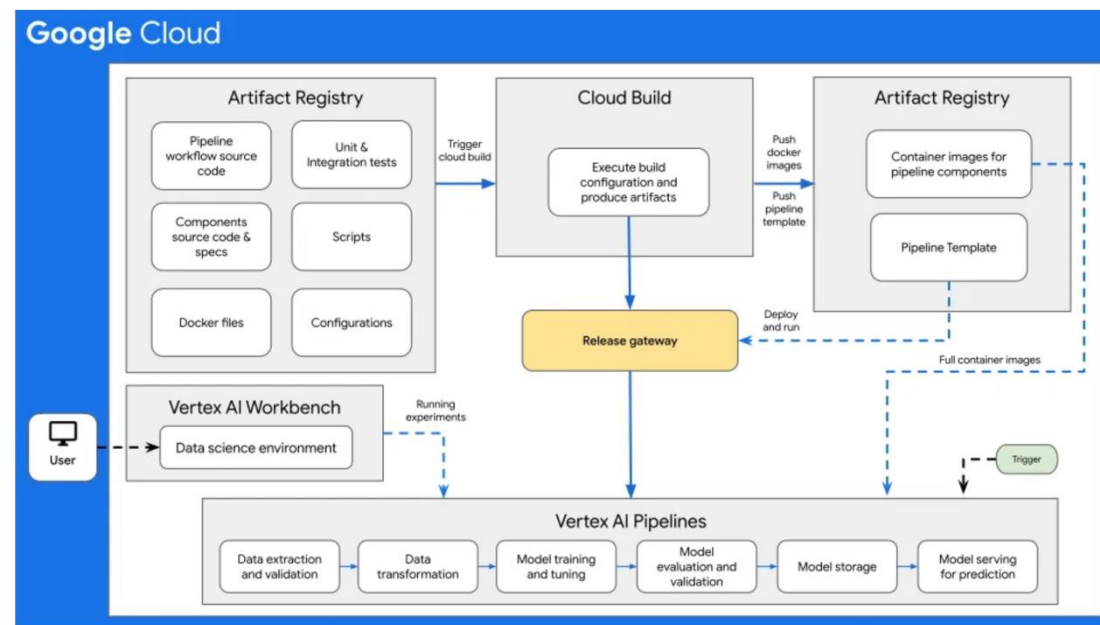
Orchestrate ML workflows with a serverless execution service based on Kubeflow Pipelines, enabling repeatable and automated ML processes.

Model Registry

Centralized repository for managing model versions, metadata, and lineage, facilitating collaboration and governance.

Model Monitoring

Detect training-serving skew and inference drift with automated alerts, ensuring model performance and reliability in production.



Vertex AI - Enterprise Features and Agent Builder

Enterprise Security

Robust security controls, encryption, and compliance certifications for regulated industries.

Vertex Explainable AI

Understand feature attribution and model decisions with transparency tools for responsible AI deployment.

Model Monitoring

Proactive monitoring for training-serving skew and inference drift with automated alerts.

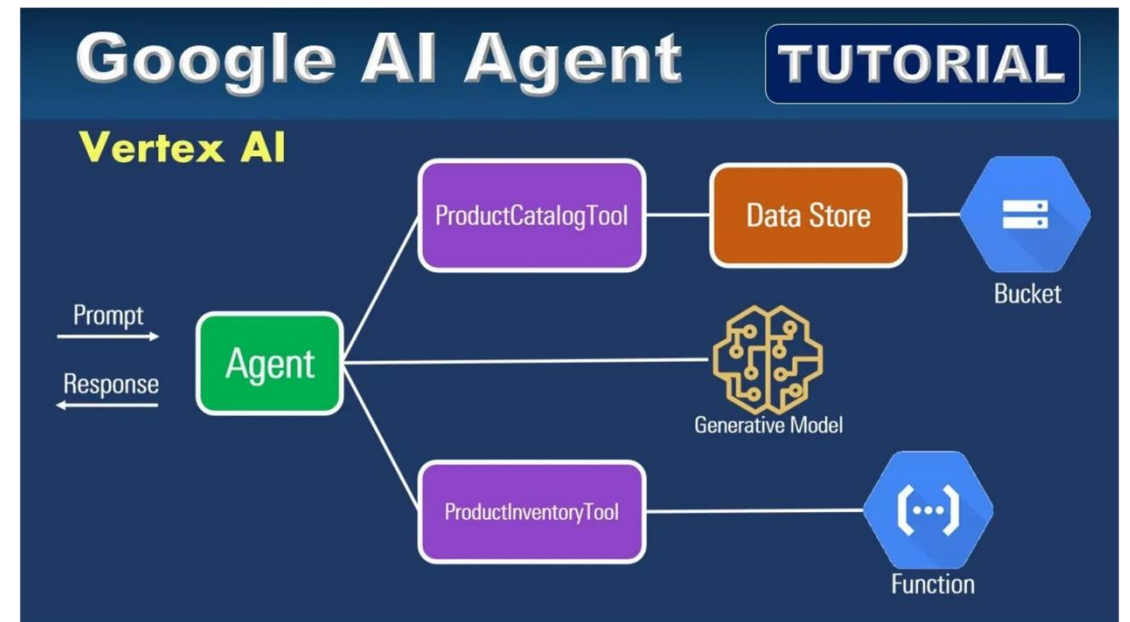
Vertex AI Agent Builder

Create enterprise-ready generative AI experiences with no-code and customization capabilities.

Build conversational AI agents grounded in organizational data

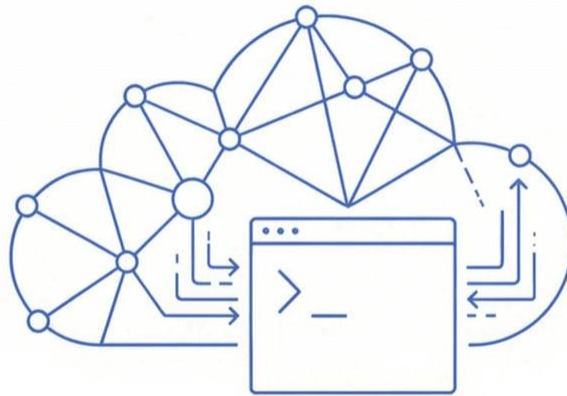
Integrate with existing business processes and systems

Deploy multi-agent experiences with orchestration capabilities



Google Cloud CLI Setup Tutorial

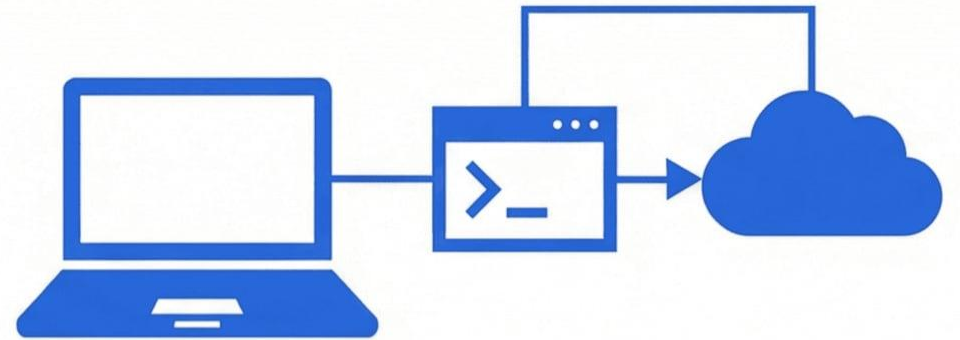
Getting Started with gcloud on macOS for Your
GenAI Class | STAT GR5293 — Spring 2026



What You Will Learn Today — Overview and Objectives

By the End of This Session, You Will Have a Fully Configured Google Cloud CLI Ready for GenAI Development

- Install and configure the Google Cloud CLI (gcloud) on macOS
- Authenticate with your Columbia/school Google account
- Connect to the class Google Cloud project
- Set up Application Default Credentials (ADC) for Python/SDK usage
- Troubleshoot common issues like expired tokens and PATH errors



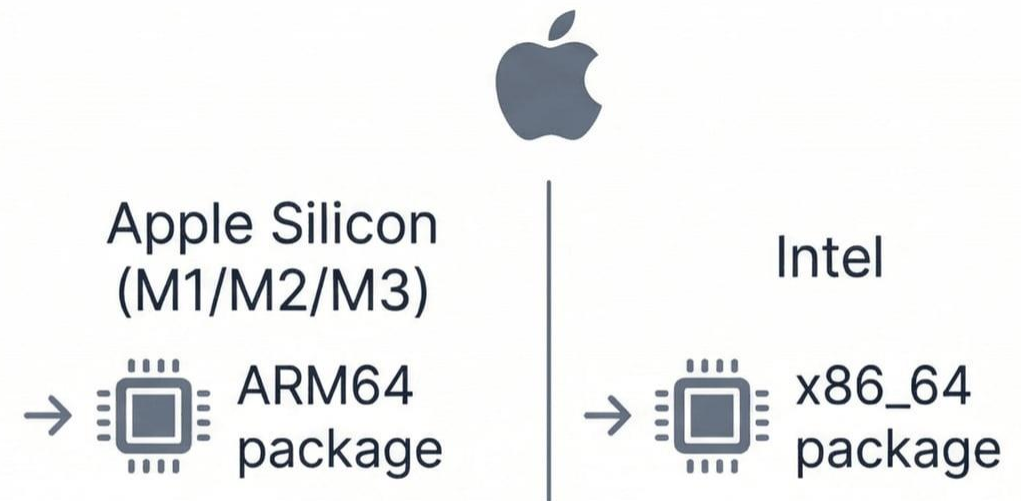
Prerequisites — What You Need Before Starting

Required Items

- A Mac with macOS (Intel or Apple Silicon)
- A Google account with access to the class GCP project
- Your Google Cloud Project ID (e.g., statg529300220261-01d2)

Know Your Chip Type

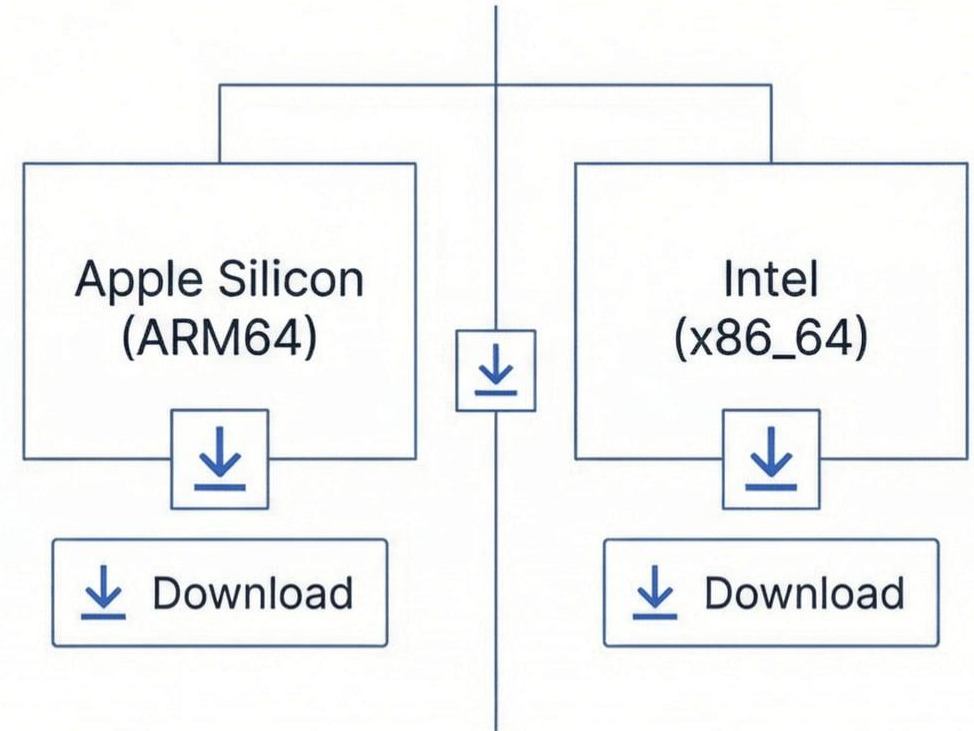
Apple menu > About This Mac



Step 1 — Download the Google Cloud CLI

Download the Correct Installer Package

- Visit cloud.google.com/sdk/docs/install
- Apple Silicon (M1/M2/M3) → Choose ARM64
- Intel → Choose x86_64
- Save .tar.gz to ~/Downloads
- Tip: Check 'About This Mac' for chip type



Step 2: Extract and Install

Extract the Archive and Run the Interactive Installer

- Open Terminal: Cmd + Space → type 'Terminal'
- Navigate: `cd ~/Downloads`
- Extract: `tar -xzf google-cloud-cli-*.tar.gz`
- Install: `./google-cloud-sdk/install.sh`
- Answer 'Y' to PATH modification & Python 3.13 install



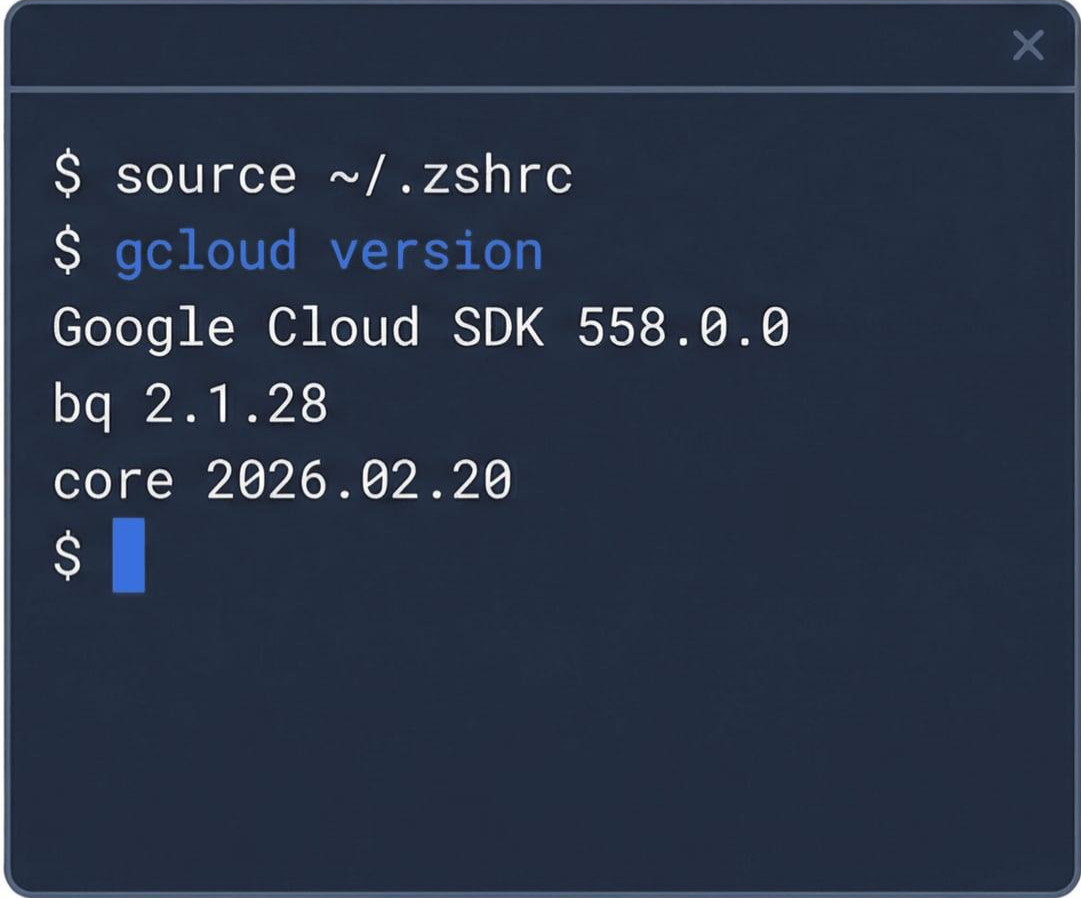
Step 3: Activate gcloud in Your Terminal

After Installation, You Must Start a New Terminal Session or Reload Your Shell Profile

The gcloud command will not be available immediately because PATH changes have not taken effect.

Key Actions

- **Option A (Recommended):** Close terminal and open a new one.
- **Option B:** Run ``source ~/.zshrc`` in current terminal.
- **Verify:** Run ``gcloud version`` to confirm installation.

A terminal window with a dark blue background and a close button in the top right corner. It shows the execution of two commands: 'source ~/.zshrc' and 'gcloud version'. The output of the second command lists the Google Cloud SDK version (558.0.0) and the versions of bq and core utilities. A blue cursor is visible on the line following the output.

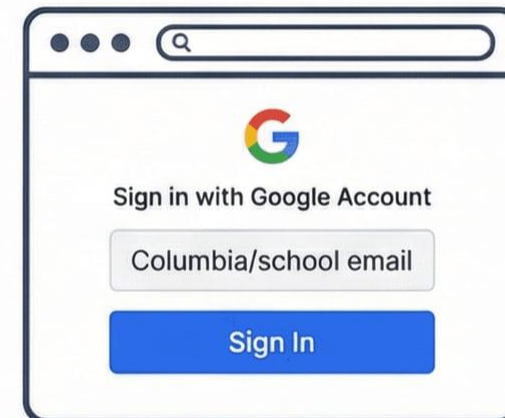
```
$ source ~/.zshrc
$ gcloud version
Google Cloud SDK 558.0.0
bq 2.1.28
core 2026.02.20
$
```

Step 4: Initialize Configuration (Part 1)

Run gcloud init to Create a Named Configuration

- Run command: `gcloud init`
- Select: [1] Create a new configuration
- Name it: `genai-class`
- Choose: [1] Sign in with a new Google Account
- **Action:** Sign in with your Columbia/school email in the browser

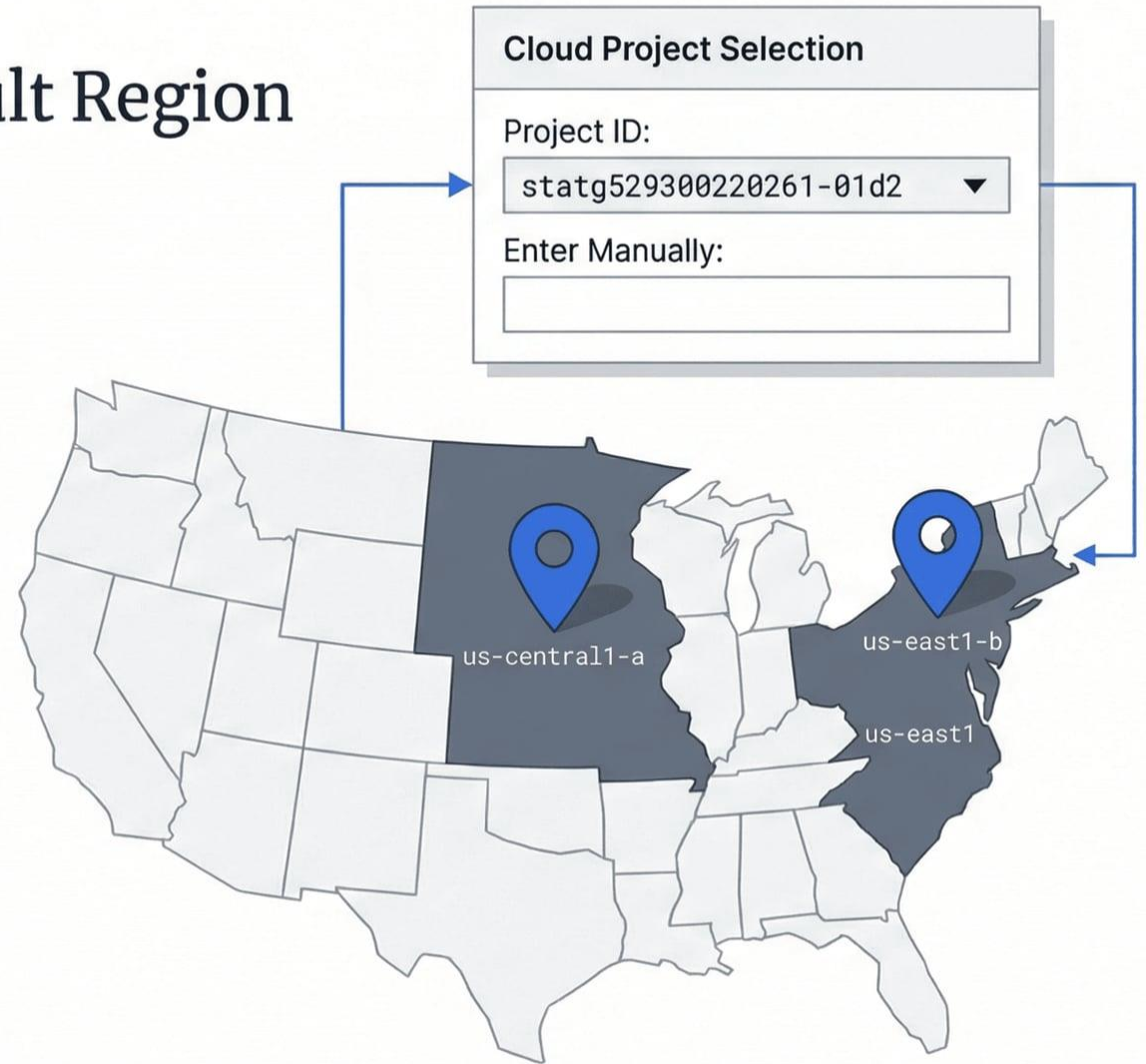
```
$ gcloud init
```



Step 4: Initialize Configuration (Part 2)

Select Your Class Project and Set Default Region

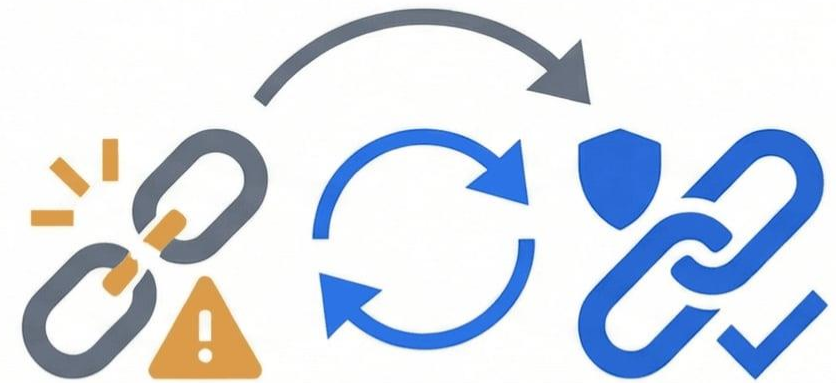
- Select or type your class Project ID (e.g., statg529300220261-01d2)
- If project not listed, enter the ID manually
- Set default region: us-central1 or us-east1
- Set default zone: us-central1-a or us-east1-b
- Note: If Compute Engine API not enabled, set defaults later



Step 5 — Fix Authentication Token Issues

If You See “invalid_grant”, Your Tokens Have Expired

- Warning: “invalid_grant: Bad Request”
- Cause: Expired or revoked credentials
- The Fix: Run `gcloud auth login`
- Action: Sign in again with school email
- Result: “You are now logged in as...”



Step 6 — Verify Your Setup with Four Key Commands



```
gcloud config get-value account
```

Expected Output: Your school email (e.g., uni@columbia.edu)



```
gcloud config get-value project
```

Expected Output: Your project ID (e.g., statg5293...)



```
gcloud config configurations list
```

Expected Output: Ensure `IS\_ACTIVE` is `True`



```
gcloud projects describe $(gcloud config get-value project)
```

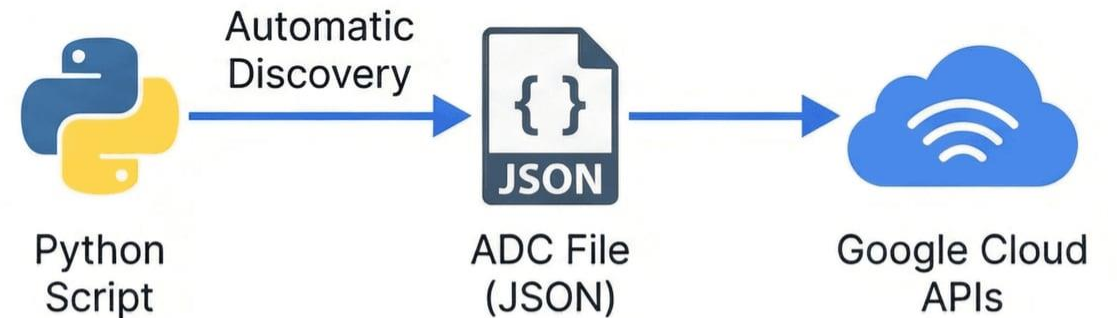
Expected Output: Project details (no errors)

Step 7 — Set Application Default Credentials for Python Development

Enable Automatic Authentication for Python SDKs

```
gcloud auth application-default login
```

- Authenticates your local environment
- Creates application_default_credentials.json
- Essential for Vertex AI & BigQuery
- Eliminates need for hardcoded API keys



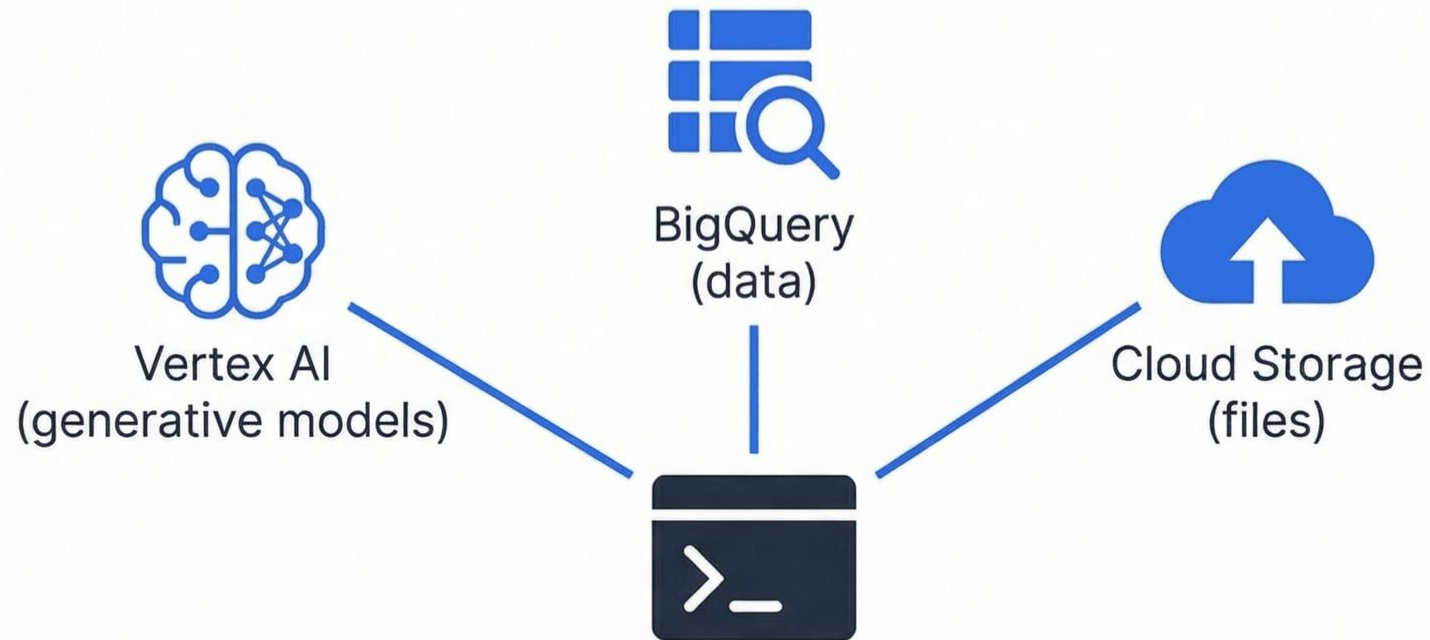
Quick Reference — Essential gcloud Commands

<u>Task</u>	<u>Command</u>
Check config	<code>gcloud config list</code>
Switch project	<code>gcloud config set project ID</code>
Switch account	<code>gcloud config set account EMAIL</code>
Re-authenticate	<code>gcloud auth login</code>
Update gcloud	<code>gcloud components update</code>
List configs	<code>gcloud config configurations list</code>

Common Issues and Troubleshooting Guide

Problem	Solution
zsh: command not found: gcloud	Open new terminal OR run source ~/.zshrc
invalid_grant: Bad Request	Run gcloud auth login
Wrong project selected	gcloud config set project [ID]
Permission denied	Check account with gcloud auth list
Python auth failure	gcloud auth application-default login

Next Steps — You Are Ready for GenAI Development



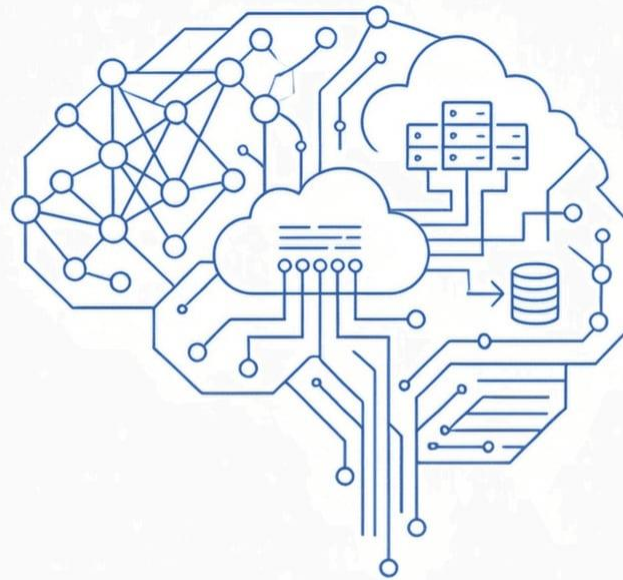
Congratulations! Setup Complete.

- Access Vertex AI & BigQuery via Python SDK
- Manage Cloud Storage & Resources
- Deploy GenAI Applications

Next Class: “First API Calls to Vertex AI”

Getting Started with Vertex AI

A Step-by-Step Guide to Setting Up and Using Gemini 3.1 Pro Preview

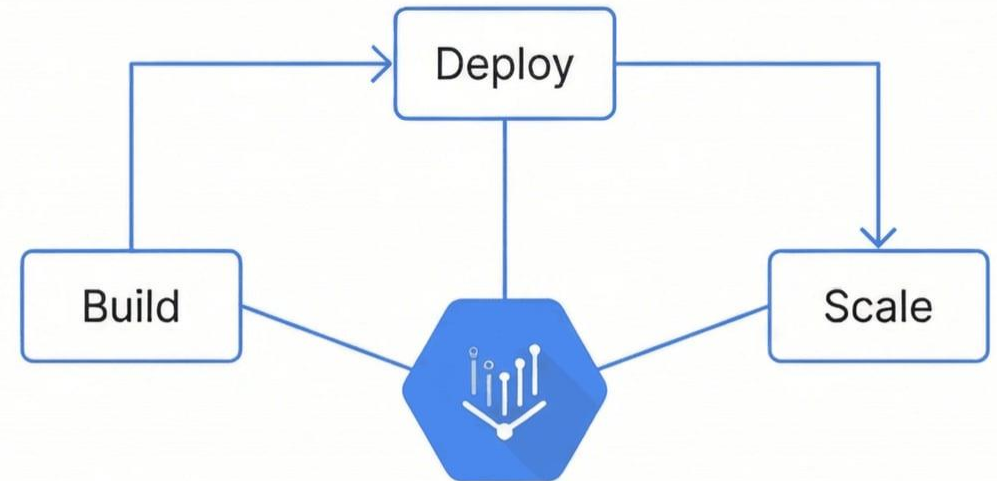


What Is Vertex AI and Why It Matters

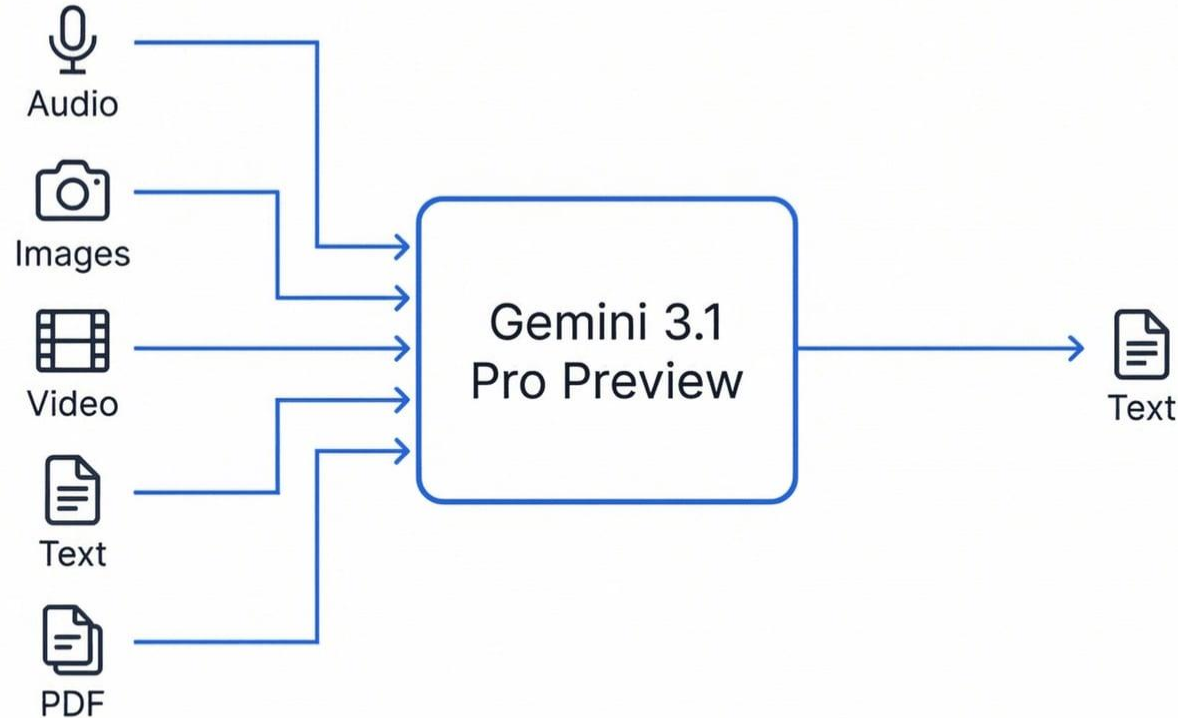
Vertex AI Provides Enterprise-Grade Access to Google's Most Powerful Generative Models

Vertex AI is a unified machine learning platform that enables users to build, deploy, and scale machine learning models with speed and efficiency. It integrates various tools and services into a single, cohesive environment, simplifying the entire ML lifecycle from data preparation to model monitoring.

- Unified ML platform for building & deploying
- Enterprise features: fine-tuning, RAG, access controls
- Gateway to Gemini models via REST & Python SDK
- Course Project ID: statg529300220261-01d2



Gemini 3.1 Pro Preview — Model Overview



1M Token Context

Thinking Mode: LOW / MEDIUM / HIGH

Model ID: gemini-3.1-pro-preview

Prerequisites — What You Need Before Starting

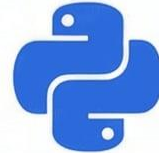
Three Prerequisites Must Be Completed Before Calling the Gemini API



gcloud CLI:
Install &
Configure
Google Cloud
CLI



Authentication:
Log in with
school email
(gcloud init)



**Python
Environment:**
Python 3.9+ &
pip3 installed

```
$ gcloud config get-value project  
> statg529300220261-01d2
```

Step 1 — Enable the Vertex AI API

The Vertex AI API Must Be Explicitly Enabled in Your Google Cloud Project

Two Methods to Enable

- **Option A:** Cloud Console → API Library → Click 'Enable'
- **Option B:** Command Line Interface (CLI)
- **Verification:** Check enabled services list
- **Note:** One-time setup per project

```
$ gcloud services enable aiplatform.googleapis.com  
Operation "operations/acf.12345" finished successfully.
```

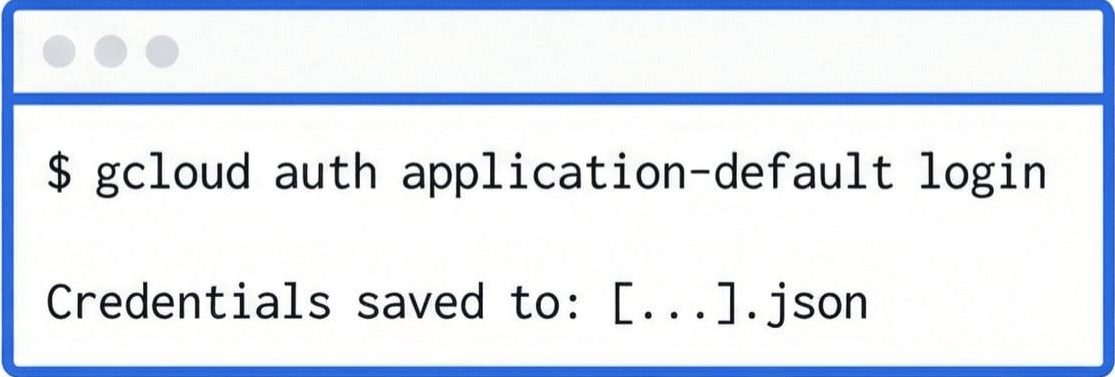
```
$ gcloud services list --enabled | grep aiplatform
```

```
aiplatform.googleapis.com    Vertex AI API
```

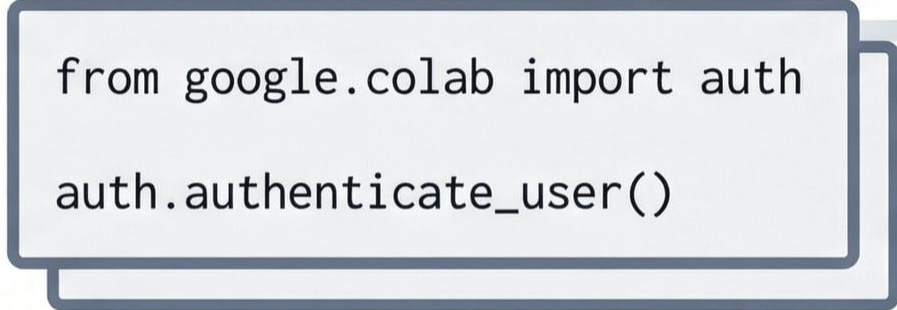

Step 2 — Authentication and Credentials Setup

Application Default Credentials (ADC)

- Unified authentication for CLI and Python
- Local: Saves to ~/.config/gcloud/...
- Colab: Built-in auth module
- **Crucial:** Sign in with school email (@columbia.edu)



```
$ gcloud auth application-default login  
Credentials saved to: [...].json
```



```
from google.colab import auth  
auth.authenticate_user()
```


Step 3 — Install the Google Gen AI SDK

The google-genai SDK Provides a Unified Python Interface

1. Install via pip

```
$ pip3 install --upgrade google-genai  
Successfully installed google-genai...
```

2. Initialize Client

```
from google import genai  
  
client = genai.Client(  
    vertexai=True,  
    project="statg529300220261-01d2",  
    location="global" # Must be global  
)
```

Location must be "global" for Gemini 3.1 Pro

Your First API Call — Text Prompt with Python

A Simple Text Prompt Requires Just Three Lines of Code

```
# 1. Define the model ID
model_id = "gemini-3.1-pro-preview"

# 2. Call the model
response = client.models.generate_content( ← 'Core Method'
    model=model_id,
    contents="Explain generative AI in 3 sentences."
)

# 3. Print the result
print(response.text) ← 'Model Output'
```

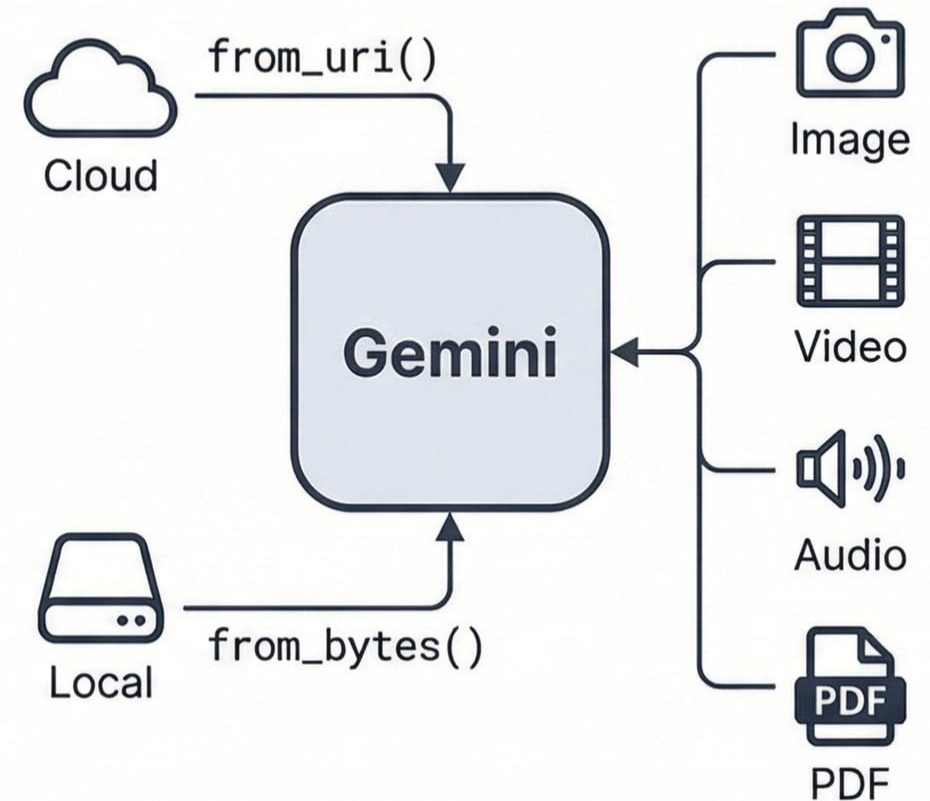
Key Concepts

- **Client:** Initialized with project & location
- **Method:**
generate_content() handles the request
- **Response:** Object containing text, usage metadata, and safety ratings

Multimodal Prompts — Sending Images to the Model

Gemini 3.1 Pro Accepts Images, Video, Audio, and PDFs Alongside Text

```
types.Content(  
    role="user",  
    parts=[  
        types.Part.from_uri(  
            file_uri="gs://bucket/img.jpg",  
            mime_type="image/jpeg"  
        ),  
        types.Part.from_text("Describe this image")  
    ]  
)
```

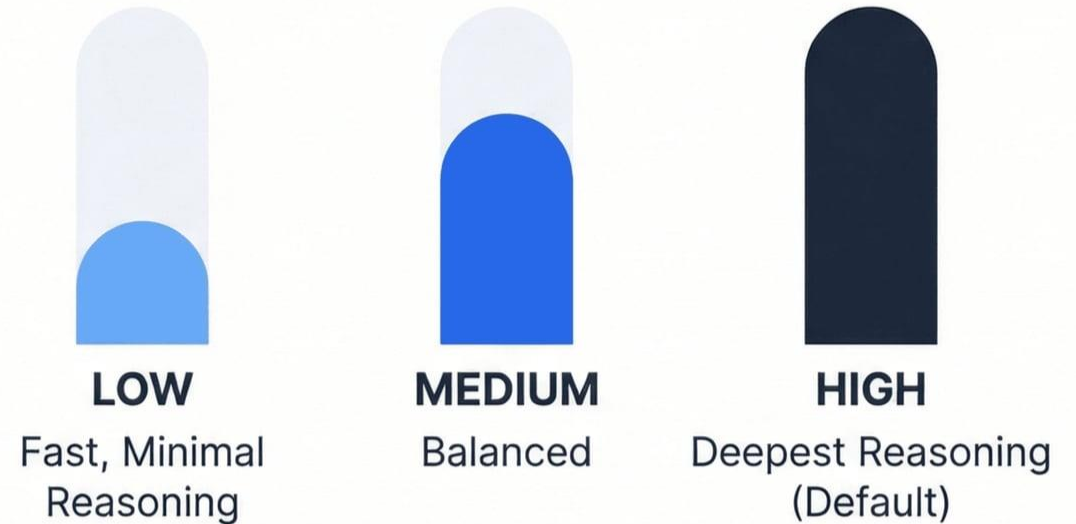


- Supports JPEG, PNG, GIF, WebP, PDF, Video
- Combine text instructions with media parts
- Use GCS URIs or raw bytes

Thinking Mode — Controlling the Model's Reasoning Depth

Trade Latency for Deeper Reasoning on Complex Problems

- Gemini 3.1 Pro reasons step-by-step
- Response includes 'thought' parts & final answer
- Inspect via `part.thought` flag
- **Warning:** Do NOT mix `thinking\_budget` & `thinking\_level`



```
config = types.GenerateContentConfig(  
    thinking_config=types.ThinkingConfig(  
        thinking_level="MEDIUM"  
    )  
)
```

Common Errors and Troubleshooting Guide

Quick Fixes for Common Setup Issues

 **Error: 'Default credentials not found'**

Fix: Run `gcloud auth application-default login`

 **Error: '403 Forbidden / Permission Denied'**

Fix: Verify Project ID & Re-authenticate with school email

 **Error: '404 Model not found'**

Fix: Check Model ID (`gemini-3.1-pro-preview`) & Location (`global`)

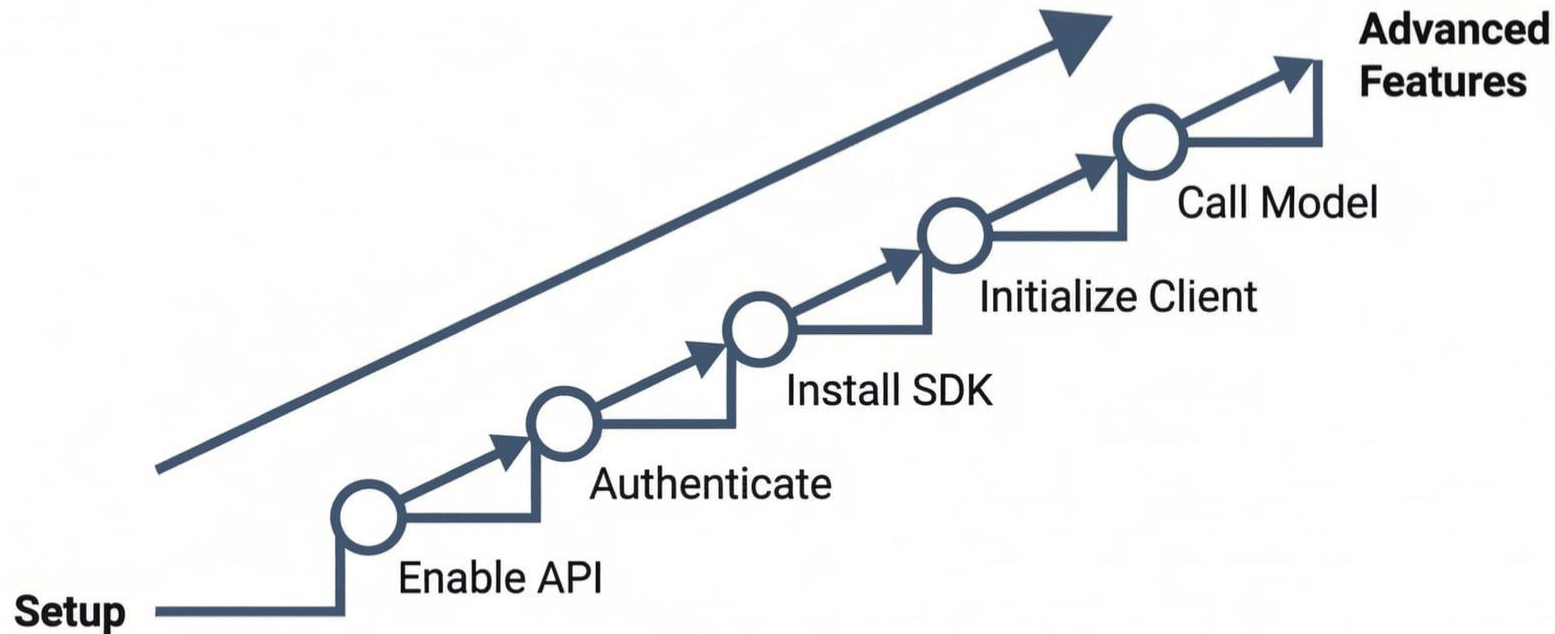
 **Error: '400 Bad Request (Thinking Mode)'**

Fix: Do NOT mix `thinking_budget` with `thinking_level`

 **Error: 'ImportError: No module named google.genai'**

Fix: `pip3 install --upgrade google-genai`

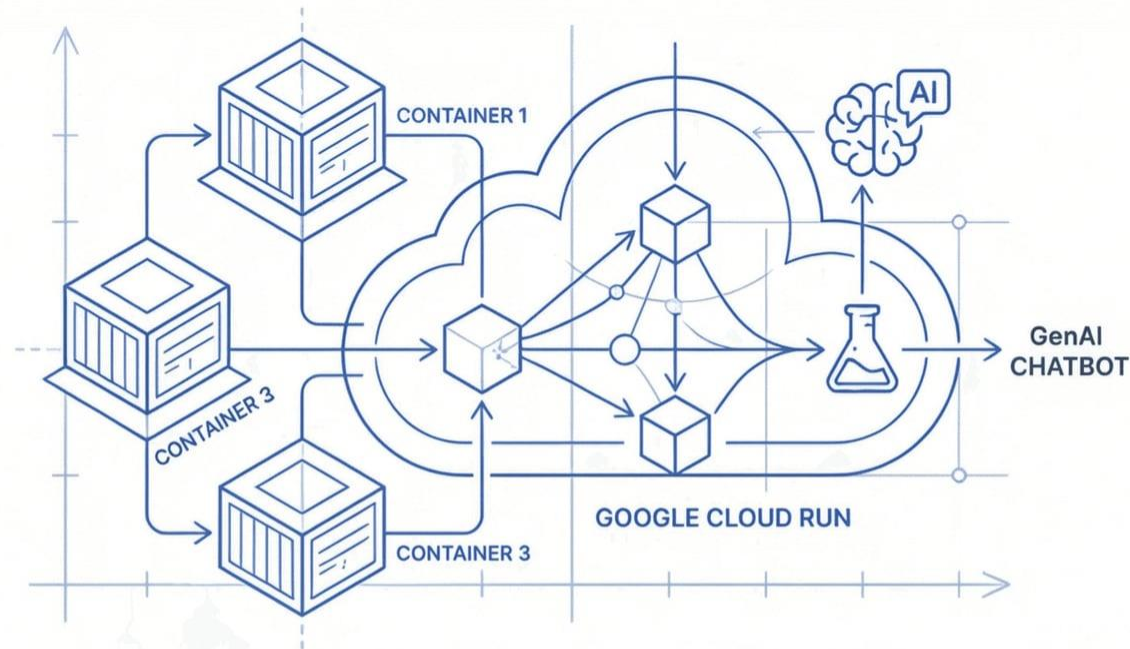
Summary and Next Steps



- **Five setup steps:** Enable API → Authenticate → Install SDK → Initialize Client → Call Model
- **Hands-on notebook:** `gemini_3_1_pro_vertex_ai_tutorial.ipynb`
- **Explore next:** Google Search grounding, RAG Engine, structured output
- **Course project:** `statg529300220261-01d2`

Deploy a GenAI Chatbot to Google Cloud Run

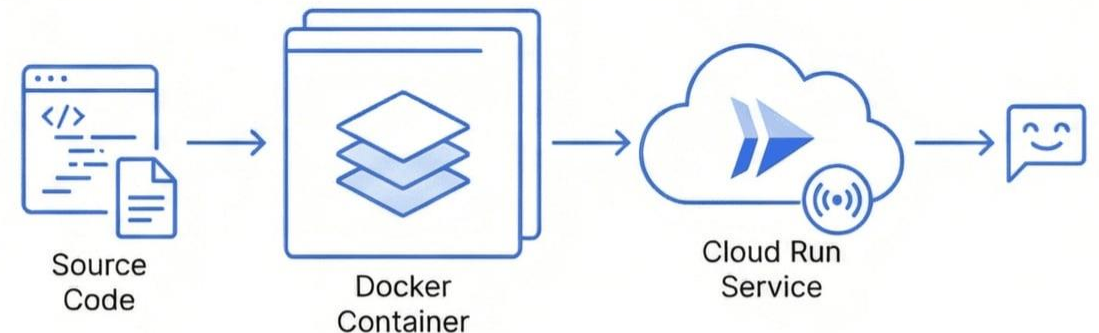
A Step-by-Step Tutorial: Flask + Gemini 3.1 Pro on Serverless Containers



What You Will Learn Today

From Source Code to a Live GenAI Chatbot in Under 10 Minutes

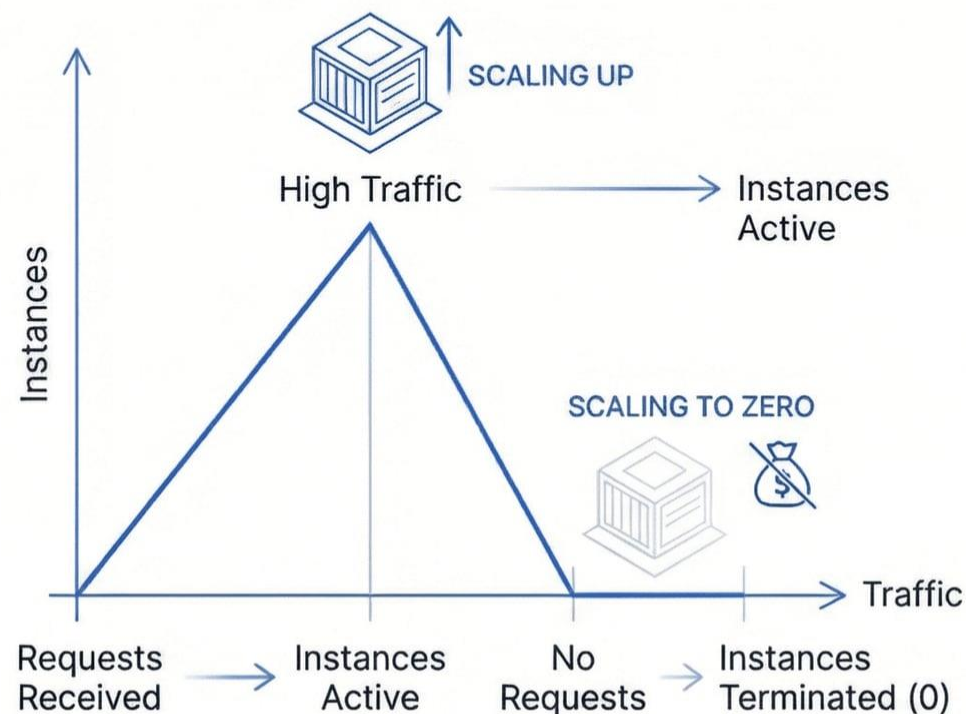
- Enable required Google Cloud APIs (Cloud Run, Cloud Build, Artifact Registry, Vertex AI)
- Write the app files: Flask backend + Gemini API + Dockerfile + Chat UI
- Deploy to Cloud Run with a single `gcloud run deploy`` command
- Configure IAM permissions for Vertex AI and service access
- Test the live endpoint via proxy, curl, or browser
- Clean up resources to avoid charges



What is Cloud Run? (Quick Recap)

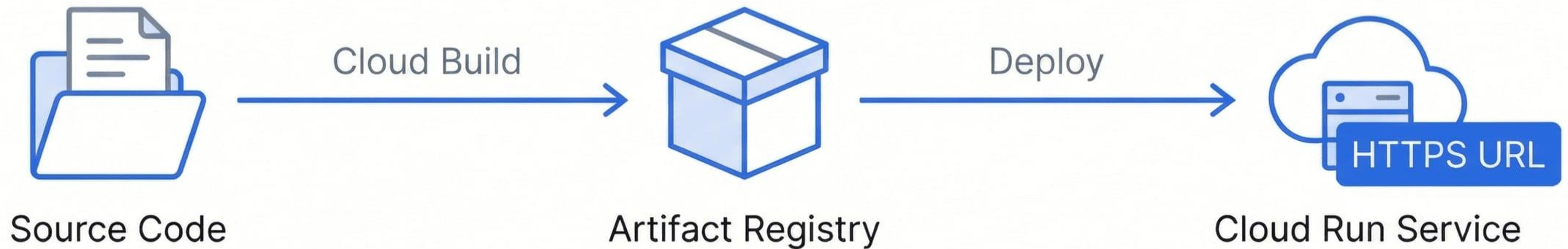
Cloud Run: Serverless Containers That Scale to Zero

- **Serverless:** No servers to manage — Google handles scaling and infrastructure.
- **Scale to zero:** You only pay when your app is handling requests.
- **Any language:** Package your app in a container and deploy it.
- **HTTPS by default:** Every service gets a unique *.run.app URL with TLS.
- **Source deploy:** Deploy directly from source code.



Architecture Overview

One Command Triggers: Build, Push, Deploy, and Serve



- Step 1: Cloud Build reads Dockerfile and builds container image
- Step 2: Image is pushed to Artifact Registry
- Step 3: Cloud Run creates a service revision and routes traffic
- Result: HTTPS URL with auto-scaling, TLS, and zero infrastructure management

Step 1: Prerequisites and Authentication

Authenticate with Google Cloud and Set Your Project

Authenticate (Colab)

```
from google.colab import auth  
auth.authenticate_user()
```



Configure Project

```
PROJECT_ID = "statg529300220261-01d2"  
gcloud config set project $PROJECT_ID
```

Verify

```
gcloud config get-value project
```

Step 2: Enable Required APIs

Four APIs Must Be Enabled Before Deployment

```
gcloud services enable run.googleapis.com cloudbuild.googleapis.com  
artifactregistry.googleapis.com aiplatform.googleapis.com
```

1. **run.googleapis.com**: Deploy and manage Cloud Run services
2. **cloudbuild.googleapis.com**: Build container images from source
3. **artifactregistry.googleapis.com**: Store container images
4. **aiplatform.googleapis.com**: Access Gemini via Vertex AI

Step 3: Grant Vertex AI Permissions

Cloud Run Uses a Service Account, Not Your Personal Credentials

Concept

Cloud Run does not use your personal credentials. It operates using a specific service account that requires explicit permissions to make calls to Vertex AI services. This ensures secure and managed access to resources.

Action

1. Find Service Account:

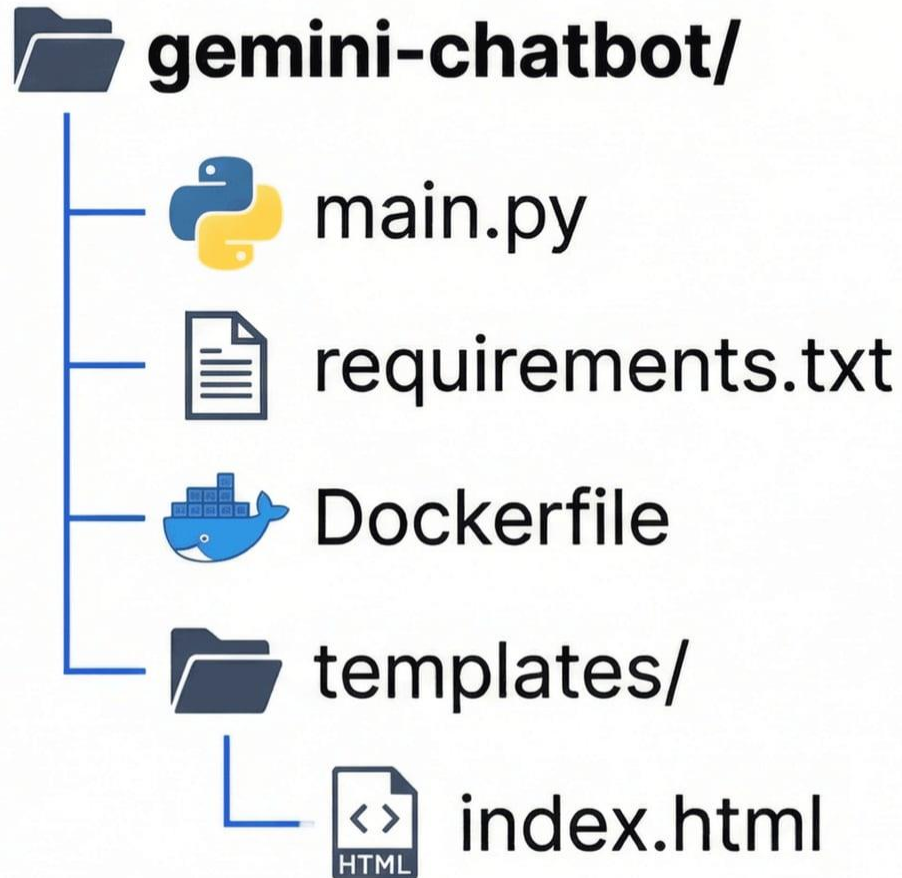
```
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")
```

2. Grant Role:

```
gcloud projects add-iam-policy-binding $PROJECT_ID --member="serviceAccount:  
${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" --role="roles/aiplatform.user"
```

Step 4: Explore the App Code

Four Files Are All You Need: main.py, index.html, requirements.txt, Dockerfile



Explanation	
main.py	Flask routes + Gemini client initialization (~30 lines)
index.html	Chat UI with message bubbles (~60 lines)
requirements.txt	
Flask, gunicorn, google-genai dependencies	
Dockerfile	
python:3.12-slim + gunicorn configuration	

Step 4 (cont.): The Flask App

Two Routes: Serve the UI and Call Gemini

- **GET / Route:** Serves the chat web UI (index.html).
- **POST /api/chat Route:** Accepts messages, calls Gemini 3.1 Pro, returns reply.
- **Vertex AI Client:** Initialized with vertexai=True for IAM-based auth (no API keys).
- **Configuration:** PROJECT_ID is injected by Cloud Run environment variables.

```
app = Flask(__name__)
client = genai.Client(vertexai=True, project=PROJECT_ID)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/chat", methods=["POST"])
def chat():
    msg = request.json.get("message")
    response = client.models.generate_content(
        model="gemini-3.1-pro", contents=msg)
    return jsonify({"reply": response.text})
```


Step 4 (cont.): Dockerfile and Requirements

The Dockerfile: From python:3.12-slim to Production-Ready Container

Dockerfile

```
FROM python:3.12-slim
ENV PYTHONUNBUFFERED=1

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD exec gunicorn --bind :${PORT:-8080} ...
```

requirements.txt

```
Flask~=3.1
gunicorn~=23.0
google-genai>=1.0
```

} Layer
Caching

Step 5: Deploy to Cloud Run

One Command: `gcloud run deploy --source .`

```
gcloud run deploy gemini-chatbot \  
  --source . \  
  --region us-central1 \  
  --set-env-vars PROJECT_ID=$(gcloud config  
get-value project)
```



--source .: Build container from current directory (no pre-build needed)

--region: Deploy to specific Google Cloud region

--set-env-vars: Inject Project ID into the container

Expected Output:

Service URL: <https://gemini-chatbot-.....run.app>

Step 6: Test the Live App

Three Ways to Test: Proxy, Curl, or Browser

Option A: Cloud Run Proxy (Recommended)

For local browser testing.

```
gcloud run services proxy  
gemini-chatbot --port  
8080
```

Open <http://localhost:8080>

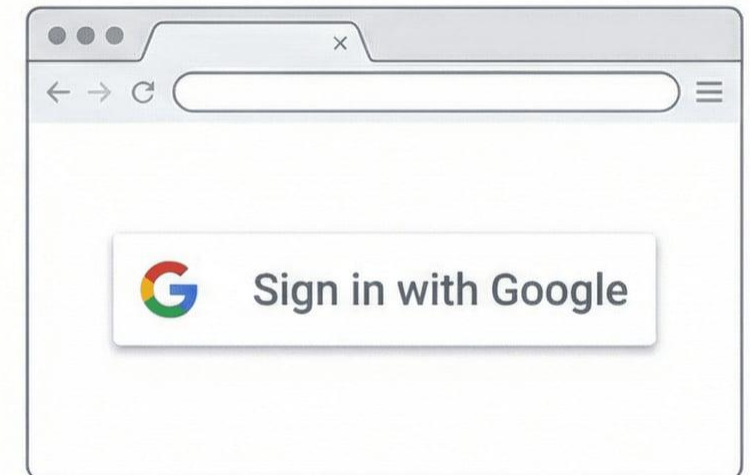
Option B: Curl with Identity Token

For API testing.

```
curl -H "Authorization:  
Bearer $(gcloud auth  
print-identity-token)" ...
```

Option C: Browser Direct Access

For users with Columbia Google accounts.



Step 7: Clean Up

Always Clean Up: Delete the Service When Done

Cloud Run charges are based on usage. Delete services you no longer need to avoid costs.

```
gcloud run services delete gemini-chatbot  
--region us-central1 --quiet
```

```
gcloud artifacts repositories delete  
gcloud artifacts repositories delete cloud-run-s  
--location us-central1 --quiet
```



Troubleshooting — Common Issues and Fixes

When Things Go Wrong: The Five Most Common Errors

Error	Cause	Fix
"Forbidden"	Auth required	Use Proxy or Domain Binding
"FAILED_PRECONDITION"	Org policy blocks public	Use domain:columbia.edu
"Permission denied"	Service account missing role	Grant roles/aiplatform.user
Build fails	Missing files	Check directory for Dockerfile
500 errors	Env vars/Permissions	Check logs

Quick Debug Command:

```
gcloud run services logs read gemini-chatbot --region us-central1 --limit 20
```


Summary and Next Steps

You Just Deployed a Production GenAI App in Under 10 Minutes

Key Takeaways

- Cloud Run = Serverless containers that scale to zero
- One command deploys: `gcloud run deploy --source .`
- Service accounts need explicit IAM roles for Vertex AI
- Cloud Run Proxy is best for local testing
- Always clean up: `gcloud run services delete`

Next Steps

- Add conversation history (multi-turn chat)
- Add streaming responses for better UX
- Connect a custom domain
- Explore Identity-Aware Proxy (IAP)
- Try deploying a different GenAI model

